FeLoG: Scalable and Efficient Distributed Graph Embedding with Feedback Loop Mechanism [Full Version]

Peng Fang[†], Arijit Khan[‡], Ziqiang Wu[†], Zhenli Li[†], Yibo Zhou[†], Fang Wang[†], Dan Feng[†]
[†]Huazhong University of Science and Technology, China [‡]Bowling Green State University, USA

ABSTRACT

Graph embedding maps graph nodes into low-dimensional dense vectors to support a wide variety of tasks such as recommendation, fraud detection, and graph-based retrieval augmented generation (GraphRAG). With graph datasets reaching billions of edges, e.g., the Stanford OGB-LSC, scalable and efficient embeddings is becoming indispensable. For scalability, existing frameworks typically adopt a sampling-training paradigm, where mini-batches are built by sampling nodes and neighbors to approximate full-graph learning. However, sampling is often decoupled from the evolving embeddings, resulting in redundant exploration of well-trained regions and insufficient attention to the undertrained nodes. Further, the same decoupling also manifests at the system level, leading to excessive communication and serialized execution which impacts the overall scalability and resource utilization in distributed settings.

In this paper, we propose FeLoG, a feedback loop-driven system for scalable and efficient distributed graph embedding, with three key technical innovations. **(1)** FeLoG introduces a feedback-coupled sampling-training model that dynamically adapts sampling using real-time embedding quality metrics. Consequently, the undertrained nodes receive higher sampling priority and thereby reduces wasted computation and accelerates convergence. **(2)** FeLoG implements an activity-aware communication mechanism to reduce communication overhead by analyzing data flow characteristics. Specifically, it compresses high-frequency node sequences to mitigate intra-machine PCIe traffic and selectively synchronize frequently updated embeddings to minimize inter-machine communication. **(3)** FeLoG adopts a round-interleaved pipeline that parallelizes sampling in the next round with training in the current round, thereby improving CPU-GPU utilization. We conduct extensive experiments on FeLoG and six state-of-the-art baselines using large-scale real-world and synthetic graph datasets. The results show that FeLoG achieves an average acceleration of 27.9× over the baselines, reduces communication cost by over 53.1%, and sustains over 80% CPU-GPU utilization, demonstrating its efficiency and scalability.

KEYWORDS

Graph embedding; Sampling-training paradigm; Feedback

PVLDB Reference Format:
Peng Fang, Arijit Khan, Ziqiang Wu, Zhenli Li, Yibo Zhou, Fang Wang, Dan Feng. FeLoG: Scalable and Efficient Distributed Graph Embedding with Feedback Loop Mechanism [Full Version]. PVLDB, 19(1): XXX-XXX, 2026.

Proceedings of the VLDB Endowment, Vol. 19, No. 1 ISSN 2150-8097.

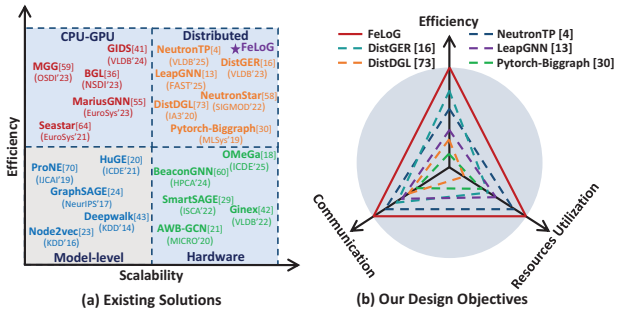


Figure 1: (a) Existing approaches vs. FeLoG (ours) w.r.t. model- and system-level design choices and efficiency-scalability performance; (b) our design objectives for the proposed system FeLoG.

PVLDB Artifact Availability:
The source code and data have been made available at https://github.com/RocmFang/FeLoG.

1 INTRODUCTION

Graph embedding [7], a powerful tool for graph analytics, has become a cornerstone for learning node representations from graph data. Specifically, it transforms high-dimensional, sparse graph structures into low-dimensional, dense vector spaces, preserving both structural and attribute information of the original graph. Beyond traditional data-driven tasks, e.g., recommendation [83], link prediction [52], and node classification [72], their versatility has recently extended to retrieval-augmented generation (RAG) [15]. Embeddings act as a bridge between graph-structured data and large language models (LLMs), enabling efficient retrieval and grounding over large-scale graphs. In such applications, graphs can be huge with millions of nodes and billions of edges. For example, the Stanford *OGB-papers100M* citation network contains over 111 million nodes and 1.6 billion edges, posing significant challenges for tasks such as link prediction and classification [27]. Similarly, GraphRAG frameworks leverage knowledge graphs with comparable or greater scale, often containing millions of entities and relationships [40].

The commonly-used graph embedding paradigms follow a two-stage process: sampling and training. In random walk-based methods [21, 24, 47], node sequences or subgraphs are sampled from the original graph, and are used to train embedding models. In graph neural network (GNN)-based approaches [25, 37, 59], the sampling step selects a subset of neighbors for each node to construct computation subgraphs, and the training stage then performs message passing and aggregation on these sampled subgraphs to learn the final embeddings. When the scale of graph data increases, the fundamental

challenges in graph embedding primarily lie in efficiency and scalability, as depicted in Figure 1(a). At the core of these challenges is the decoupled sampling-training paradigm, which serves as the root cause of inefficiencies across both the model and system levels.

From the model perspective, the separation of sampling from training generates large amounts of redundant or uninformative training data without guidance from the evolving embedding states, thereby driving both computational and memory inefficiencies. Classical methods such as DeepWalk [47] and node2vec [24] require months to process graphs with hundreds of millions of nodes and edges [79], and their scalability is further constrained by excessive memory consumption [21]. Even neighbor-sampling methods such as GraphSAGE [25] spend over 80% of runtime on sampling due to the lack of feedback from training. More recent methods like HuGE [20, 21] guide sampling adaptively but remain limited by the decoupled paradigm, hindering redundancy control and efficiency.

From the system perspective, existing efforts exploit heterogeneous architectures or distributed clusters by leveraging stronger GPUs [39, 60, 68], larger SSDs [32, 45, 46], or scale-out designs [17, 65, 85]. However, the decoupled sampling-training paradigm is also the key reason behind system-level inefficiencies, which persist despite these advances. One critical challenge is high communication overhead, as the bandwidth gap between computation and interconnect severely limits scalability. Since sampling is decoupled from training, the system must transfer all sampled data, including redundant neighborhoods, across CPU-GPU or machine boundaries, which significantly inflates communication costs. For example, the NVIDIA A100 provides 2 TB/s of GPU memory bandwidth, whereas PCIe bandwidth remains around 20 GB/s, making CPU-side sampling the bottleneck. In distributed settings, synchronizing embeddings for billion-edge graphs such as *Twitter* [31] requires hundreds of GBs of data exchange, incurring multi-second latencies and reducing GPU utilization to as low as 10% in DistDGL [85]. Another challenge is low resource utilization, since the sequential dependency enforced by decoupled sampling and training prevents overlap between the two stages and results in long idle periods. In DistGER [17], CPU and GPU utilization remain only 15.8% and 25.5%, respectively, even under heavy workloads. Pipeline and caching strategies adopted in MariusGNN [60] and Ginex [46] partially mitigate this issue, but their decoupled designs lack runtime feedback to dynamically adapt sampling to training needs.

To address the above challenges, we present FeLoG, a distributed system built around a sampling-training **Fe**edback **Lo**op for **G**raph embedding. This feedback loop directly targets the root cause of inefficiencies and serves as the foundation that enables our three technical contributions. Figure 1(b) summarizes the design objectives of FeLoG, which correspond to improving execution efficiency, reducing communication overhead, and enhancing resource utilization. **First**, FeLoG introduces a Feedback-coupled Sampling-Training model (FeeST) that tightly integrates sampling with training. Unlike existing methods that sample blindly or heuristically, FeeST leverages real-time feedback from embedding quality to guide subsequent sampling. By prioritizing nodes with undertrained embeddings, it captures more informative context, reduces redundancy, and accelerates convergence. **Second**, to mitigate communication bottlenecks, FeLoG incorporates an Activity-aware Communication Mechanism (ACM). Here, activity refers to dynamic access and update patterns of nodes during embedding training, e.g., how frequently a node or its context is sampled and how often its embedding is modified. By exploiting these patterns, ACM analyzes data flow characteristics and applies targeted optimizations. Specifically, it uses a Frequency-aware Bitmap Compression strategy (FaBC) to reduce PCIe traffic between CPU and GPU, and a Hotspot-aware Synchronization strategy (HaSyn) to minimize inter-machine communication by focusing only on embeddings with high update activity. **Last**, to overcome resource underutilization, FeLoG employs Round-interleaved Pipeline Parallelism (RiPP), which overlaps sampling in the next round with training in the current one. This design decouples strict sequential execution, allowing CPU-based samplers and GPU-based trainers to operate concurrently across rounds, improving system throughput.

Contributions and roadmap. We present FeLoG, an efficient, scalable, end-to-end distributed graph-embedding system, which, to our best knowledge, is the first general-purpose system that integrates feedback loop coordination to couple sampling and training.

- We propose the feedback-coupled sampling-training model (FeeST) for adaptive sampling based on embedding quality (§5.1).
- We develop an activity-aware communication mechanism (ACM), including frequency-aware bitmap compression (FaBC) and hotspot-aware synchronization (HaSyn), to reduce intra- and inter-machine communication overheads (§5.2).
- We design the round-interleaved pipeline parallelism (RiPP) to overlap sampling and training, improving resource utilization (§5.3).
- We demonstrate the generalization of FeLoG, supporting diversity techniques as well as diverse graph types, including purely structural graphs and structure with feature graphs (§5.4 & §6).
- We conduct extensive experiments on seven large, real-world and synthetic graphs to demonstrate that FeLoG achieves superior efficiency, scalability, and effectiveness over state-of-the-art, popular distributed graph embedding frameworks. Moreover, we are the first to exhibit a Graph-based RAG case study using distributed graph embeddings, showcasing FeLoG's integration into a retrieval-augmented generation pipeline (§6).

2 PRELIMINARIES

Most graph embedding methods use a two-stage pipeline: a sampling stage that extracts node sequences or subgraphs from the input graph, followed by a training stage that learns node embeddings from those samples (Figure 2). The stages run independently, with sampling driven only by the graph structure or previous samples and receiving no feedback from the training process.

Sampling process is commonly implemented via truncated walks. Starting from each node, multiple fixed-length walks are performed to explore the local graph structure. Walk directions are determined based on structural heuristics, such as node degree or common neighbors [21]. To ensure coverage, sampling is repeated across the entire graph. While effective, this process often generates redundant or low-quality information, especially when applied uniformly across all nodes regardless of their structural roles [47]. For GNN-based approaches, the sampling step corresponds to selecting a subset of neighbors for each node at every layer, thereby constructing a smaller computation subgraph for subsequent training. This reduces the input size compared to using the full neighborhood, but redundancy still exists when large neighborhoods are uniformly sampled.

Table 1: Frequently used notations.

Notation	Meaning
$G = (V, E)$	Undirected, unweighted graph: V nodes and E edges
$\varphi(u)$	Embedding of node u with dimension d
$\mathcal{N}(v)$	Sampled nodes accessed for message passing of node v
V_a^r	Active nodes in round r
$\Psi(v)$	Embedding quality of node v
L	Random walk length
l	l-th GNN layer
r	Number of random walks per node

Training process typically uses models inspired by natural language processing, such as word2vec [44]. Here, sampled node sequences form a corpus, with nodes as words and their co-occurrences defining context. The Skip-gram model predicts surrounding nodes (context) given a target node (center), learning embedding vectors that capture structural and semantic proximity. Formally, it maximizes the co-occurrence probability within a window w:

$$\underset{\tilde{V}}{\operatorname{argmax}} \frac{1}{|V|} \sum_{j=1}^{|V|} \sum_{-w \le i \le w} \log p(u_{j+i}|u_j) \tag{1}$$

The generated walks are treated as a corpus with vocabulary V, where u_{j+i} denotes a context node in a window w. Existing methods speed-up training via negative sampling [43]. It adjusts the embeddings such that the target node is close to its true context nodes in the vector space, while the negative samples are pushed farther away.

R1.D1

$$\begin{aligned} \log p(u_j|u_{j+i}) \approx & \log \sigma(\varphi_{in}(u_{j+i}) \cdot \varphi_{out}(u_j)) \\ & + \sum_{k=1}^{K} \mathbb{E}_{u_k \sim Pn(u)} [\log \sigma(-\varphi_{in}(u_{j+i}) \cdot \varphi_{out}(u_k))] \end{aligned} \tag{2}$$

Here, $\sigma(x) = \frac{1}{1+exp(-x)}$ is the sigmoid function, and the expectations are computed by drawing random nodes from a sampling distribution $Pn(u)$, $\forall u \in V$. Typically, the number of negative samples K is much smaller than $|V|$ (e.g., $K \in [5, 20]$). However, since the training process does not influence how samples are collected, it cannot correct or avoid redundant or ineffective sampling.

For GNN-based methods, the training stage performs iterative message passing and aggregation on the sampled subgraphs. Each node updates its embedding by aggregating the representations of its sampled neighbors through parameterized functions (e.g., mean pooling, attention), followed by non-linear transformations. Formally, the l-th layer of a GNN is expressed as:

$$h_v^{(l)} = \sigma\Big(W^{(l)} \cdot \mathrm{AGG}\big(\{h_u^{(l-1)} : u \in \mathcal{N}(v) \cup \{v\}\}\big)\Big) \tag{3}$$

where $h_v^{(l)}$ is the embedding of node v at layer l, $\mathcal{N}(v)$ are the sampled neighbors, $\mathrm{AGG}(\cdot)$ is a permutation-invariant function, $W^{(l)}$ is a learnable weight matrix, and $\sigma(\cdot)$ is a non-linear activation.

Complexity analysis. For random walk-based approaches, assume that the number of walks per node is r, walk length L, embedding dimensions d, window size w, and the number of negative samples K. The time complexity of sampling procedure is $O(r \cdot L \cdot |V|)$. For training procedure, the corpus size $C = r \cdot L$. Let us denote the complexity of the unit operation of predicting and updating one node's embedding as o. The Skip-Gram with the negative sampling only needs $K + 1$ words to obtain a probability distribution (Eq. 2), thus the time complexity of training procedure is $O(C \cdot w \cdot (K+1) \cdot o)$. For GNN-based methods, let $|\mathcal{N}(v)|$ denote the total number of nodes accessed for message passing for node v across all l layers (including multi-hop neighbors selected by sampling). The sampling complexity is $O(|V| \cdot |\mathcal{N}(v)|)$, and the training complexity from Eq. 3 (i.e., one forward pass in GNN) is $O(|V| \cdot |\mathcal{N}(v)| \cdot d)$.

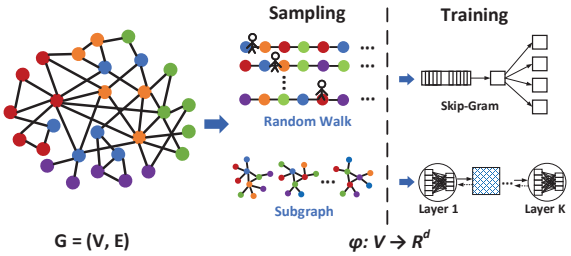


Figure 2: Sampling and training in graph embedding.

3 RELATED WORK

We analyze existing graph embedding systems from both single-machine and distributed perspectives, with additional discussions of model-level graph embedding methods provided in Appendix A.

Graph embedding algorithms. Graph embeddings [7] have been applied to uncertain graphs [26], dynamic graphs [67], heterogeneous networks [66], retrieval-augmented generation [82], and knowledge graphs [9]. Broadly, recent algorithms fall into two families. *GNN-based* approaches [25, 58, 59, 63] learn embeddings via iterative message passing and aggregation over sampled neighborhoods, enabling semi-supervised or supervised learning; representative methods include GraphSAGE [25], GAT [59], and GraphGAN [63]. *Random-walk-based* methods [20, 21, 24, 47] transform graphs into node sequences via random walks and train with Skip-Gram [44]; e.g., DeepWalk [47] pioneered the idea and node2vec [24] introduced biased walks, while HuGE [20, 21] adapts sampling configurations on the fly. Despite their differences, both paradigms adopt a sequential, decoupled execution of sampling and training, which can induce redundant sampling and high training overhead, ultimately limiting efficiency and scalability [17].

Single-Machine Graph Embedding Systems. To overcome efficiency bottlenecks in large-scale graph embedding, prior work ranges from full-GPU systems (e.g., GENTI [77], GAMMA [50], and XGNN [55]) that maximize on-device throughput to heterogeneous architectures that address end-to-end bottlenecks. Early systems such as BGL [39] adopt a CPU-GPU hybrid design, with CPUs performing random-walk sampling and GPUs handling embedding training. MariusGNN [60] further optimizes this paradigm with disk-efficient pipelines, while Seastar [73] provides a GPU-centric execution framework with a vertex-centric programming model [41]. Recently, NeutronOrch [2] orchestrates computation and communication across devices via runtime scheduling, DAHA [36] adapts hardware assignment based on workload characteristics, and GIDS [45] enables GPU-direct storage access to accelerate data loading. LPS-GNN [14] further scales single-node GNN training via improved partitioning. Despite these advances, single-machine systems still face fundamental scalability limits, primarily due to CPU-GPU imbalance and the constrained GPU memory capacity. Beyond CPU-GPU pipelines, several systems leverage specialized hardware to further enhance scalability and efficiency. Ginex [46]

Table 2: Comparison of distributed graph embedding systems.

System	RW	GNN	Coupling	Commun.	Pipeline
DistDGL [85]	✗	✓	✗	*Partial*	✗
PBG [33]	✗	✗	✗	✗	✗
ByteGNN [84]	✗	✓	✗	✓	✗
DistGER [17]	✓	✗	✗	*Partial*	*Partial*
LeapGNN [13]	✗	✓	✗	✓	✗
NeutronHeter [8]	✗	✓	✗	✓	*Partial*
NeutronTP [3]	✗	✓	✗	✓	✗
FeLoG **(ours)**	✓	✓	✓	✓	✓

exploits SSDs to extend memory for large-scale embeddings; SmartSAGE [32] and AWB-GCN [22] design hardware-conscious scheduling strategies for GNN workloads; BeaconGNN [69] incorporates hardware accelerators for higher throughput; and the very recent OMeGa [19] utilizes heterogeneous memory processing to deliver high-performance embedding at scale. While these systems train on larger datasets, they often sacrifice generalizability and face compatibility challenges across diverse heterogeneous environments.

Distributed Graph Embedding Systems. To scale graph embeddings, distributed systems have been developed to leverage multi-machine and multi-GPU clusters. MSPipe [53] accelerates memory-based temporal GNN training on multi-GPU platforms via staleness-aware pipeline scheduling, focusing on temporal memory dependencies that are orthogonal to FeLoG's feedback-driven sampling-training coordination. GraNNDis [54] accelerates distributed GNN training on multi-server clusters by reducing inter-server communication, redundant computation, and sampling-induced inefficiency. NuPS [51] addresses two major sources of non-uniform parameter accesses during training, improving efficiency of distributed parameter management. Amazon's DistDGL [85] extends the Deep Graph Library [64] with distributed GNN training via mini-batch execution. Similarly, PyTorch-BigGraph [33] scales embedding training to large graphs using graph partitioning and parameter servers. However, both face communication bottlenecks that limit GPU utilization and scalability (§6). Recent works enhance distributed efficiency via improved scheduling and communication. ByteGNN [84] employs two-level scheduling and tailored graph partitioning to improve parallelism. DistGER [17] and DistGER-Pipe [18] adopt information-oriented random walks to reduce redundant sampling and communication. LeapGNN [13] proposes a feature-centric distributed GNN training framework to optimize feature access and communication. However, these systems largely suffer from decoupled execution and high transfer overheads under heterogeneous clusters. NeutronHeter [8] accelerates GNN training through hierarchical workload mapping and adaptive communication, while NeutronTP [3] optimizes tensor parallelism for GNNs by improving communication and memory usage. However, NeutronTP applies uniform processing across all node features and lacks feedback-driven embedding evaluation, leading to redundant computation and slower convergence compared to FeLoG (§6). Table 2 compares representative distributed graph embedding systems across two techniques: random walk-based (*RW*) and GNN based embeddings(*GNN*), along with three system-level dimensions: sampling-training coupling (*Coupling*), communication optimization (*Commun.*), and pipeline optimization (*Pipeline*). Existing systems typically address only a subset, while FeLoG unifies all

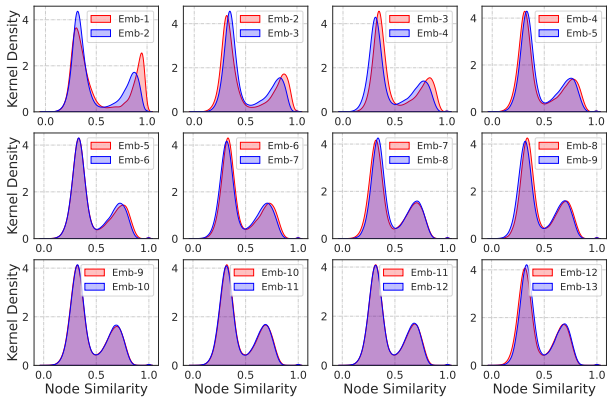


Figure 3: Kernel density distribution of node embedding similarity across different sampling-training rounds on *Flickr* graph. Each subplot compares the similarity distributions of embeddings from round i (red) and round $i+1$ (blue). The x-axis represents cosine similarity, the y-axis represents density.

embedding paradigms with comprehensive system-level optimizations.

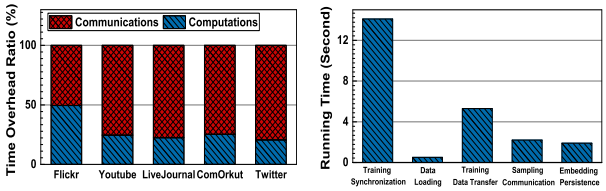
4 MOTIVATIONS

We design a scalable and efficient distributed graph-embedding system that couples sampling and training through a feedback loop. To motivate this, we first analyze the limitations of the decoupled sampling-training paradigm and identify system-level bottlenecks that hinder efficiency and scalability in distributed settings.

4.1 Limitations of the Sampling-Training Decoupled Paradigm

Despite efforts to improve sampling quality, existing graph embedding methods still operate under the decoupled sampling-training paradigm, limiting efficiency and scalability on large graphs. For example, node2vec [24] uses fixed configurations for biased random walks (e.g., walk length $L = 80$, $r = 10$ random walks per node), effective for smaller graphs but generating redundant walks for large-scale graphs. To address this, HuGE [21] introduces an information-theoretic random walk mechanism that adaptively determines walk configurations. While reducing redundancy, it still adheres to a sequential, decoupled sampling-training design, resulting in slow training times. For instance, processing a billion-edge *Twitter* graph [31] takes over a week on a modern server. The most recent system, DistGER [17], extends HuGE with incremental information-centric computation in distributed environments, but still evaluates the quality of sampled information solely based on historical statistics, leading to redundancy in sampling. In our observation, DistGER spends over 50% of its total execution time on sampling, with training contributing only 27%, demonstrating inefficiency in the decoupled execution paradigm.

We evaluate whether heuristic-driven sampling strategies like HuGE and DistGER effectively capture informative data without redundancy. Using the *Flickr* graph [56] and following the default configuration of DistGER, which heuristically determines the r by comparing the distribution of node occurrences in sampled walks with the node degree distribution. In our experiment the sampling procedure terminated after 12 rounds. After each sampling-training



(a) Runtime Breakdown (b) Communication Type

Figure 4: Analysis of communication overheads.

round, we compute the pairwise cosine similarity of node embeddings and use kernel density estimation [71] to obtain the distribution of these similarity values. Figure 3 visualizes how these distributions evolve, where each subplot compares consecutive rounds. We observe that the distributions converge around the 6th round, with minimal changes in subsequent rounds. This indicates that the remaining sampling rounds introduce little additional contribution, demonstrating redundancy in both computation and communication.

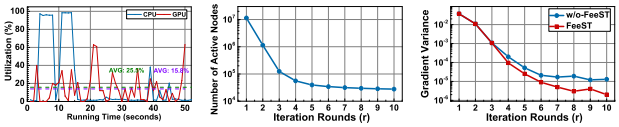
Current methods make sampling decisions based on structural heuristics or historical statistics, which are indirect proxies for representation quality and do not reveal whether the samples is effectively utilized. A mechanism is needed to identify converged nodes and those that remain undertrained, enabling efficient sampling. Without runtime feedback, samplers continue redundant exploration. This highlights the importance of training feedback for eliminating redundancy and enabling efficient, scalable graph embedding.

4.2 System-Level Bottlenecks in Distributed Graph Embedding

To address scalability challenges, existing graph embedding systems have been extended to distributed settings and CPU-GPU heterogeneous architectures. In these systems, sampling is typically executed on CPUs, while training is offloaded to GPUs. However, such designs still suffer from significant system inefficiencies. To systematically analyze these bottlenecks, we implemented DistGER on an 8-machine cluster with a CPU-GPU heterogeneous architecture.

Data Communication Overhead. We conducted experiments on five real-world datasets and profiled the end-to-end runtime of DistGER, decomposing it into computation (sampling and training) and communication (both intra- and inter-machine). As shown in Figure 4(a), communication dominates the total runtime, accounting for an average of 71%, with its proportion increasing as the dataset size grows. We further analyzed communication costs using the *Flickr* dataset by categorizing the communication into distinct types (Figure 4(b)). Among them, training synchronization and training data transfer emerge as major performance bottlenecks.

During training synchronization, updated embedding gradients must be synchronized across machines over Ethernet to ensure consistency. The communication volume increases linearly with the number of machines. For instance, in our 8-machine setup, synchronizing 128-dimensional embeddings on the *Twitter* graph using 10 Gbps NICs may introduce up to 4.6 seconds of delay per round, causing communication to dominate the runtime and accounting for 78% of the total execution time. Training data transfer also incurs considerable cost. Since sampling is decoupled from training, all sampled data including redundant neighborhoods must be moved from CPU memory to GPU memory and persisted to storage. This



(a) CPU/GPU Utilization (b) Active Nodes (c) Gradient Variance

Figure 5: Resource utilization in decoupled sampling-training, and active nodes and gradient variance on *YouTube* graph.

overhead grows proportionally with the size of training data and is constrained by PCIe and I/O bandwidth.

Decoupled Execution of Sampling and Training. In DistGER, sampling and training follow the sampling-training decoupled paradigm and are separated by a round-level barrier: training cannot begin until sampling fully completes and materializes the node sequences for the current round. Once sampling finishes, the resulting node sequences are passed to the training stage as input. To ensure fault tolerance, DistGER checkpoints the sampled data to storage before launching training, introducing additional I/O overhead. As a result, sampling and checkpointing phase lies on the critical path; any imbalance or delay in sampling directly stalls GPU execution, creating bubbles in the end-to-end pipeline. Figure 5(a) shows that CPU and GPU are significantly underutilized on the *YouTube* graph, with average utilizations of only 15.8% and 25.5%, respectively.

5 FeLoG

To address the challenges from sampling-training decoupled paradigm and system-level bottlenecks, we design FeLoG, a scalable and efficient distributed graph embedding system that integrates feedback loop coordination to couple sampling and training. Figure 6 summarizes the workflow of FeLoG. We discuss our feedback-coupled sampling-training model (FeeST) in §5.1, and activity-aware communication mechanism (ACM) in §5.2, including the frequency-aware bitmap compression strategy (FBC) and the hotspot-aware synchronization strategy (HaSyn), while our novel round-interleaved pipeline (RiPP) is given in §5.3. Finally, we present how FeLoG can be integrated into existing graph embedding frameworks in §5.4.

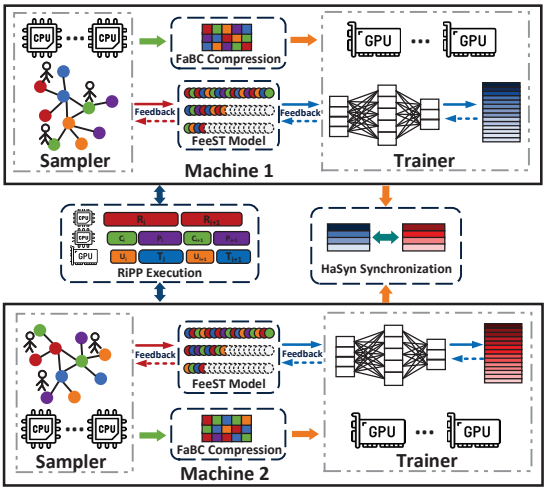


Figure 6: The workflow of FeLoG.

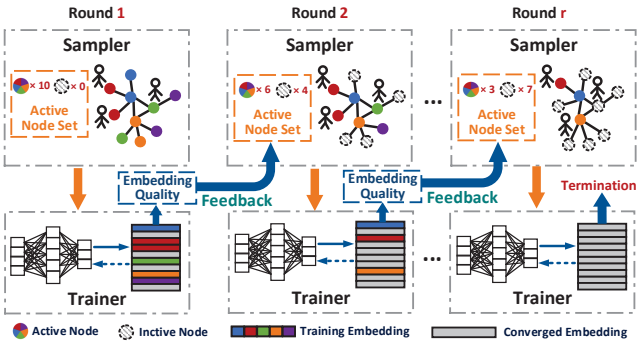


Figure 7: Feedback-driven sampling-training coupling model.

5.1 Feedback-coupled Sampling-Training Model

Traditional graph embedding pipelines treat sampling and training as separate stages, which often leads to redundant sampling and unnecessary computation on large-scale graphs (§4.1). Existing fixes [17, 21] decide per-node sampling rounds using structural heuristics or historical statistics, yet lack training feedback and may still incur substantial redundancy. To overcome this limitation, we propose the *Feedback-coupled Sampling-Training Model* (FeeST), which integrates real-time training feedback into sampling. The key idea is to derive a neighborhood-consistency metric from training feedback as a direct proxy for representation quality, so that only under-trained nodes remain active for subsequent sampling.

Overview of FeeST**.** As illustrated in Figure 7, FeeST closes the loop between sampling and training via an *active node set*. At round r, FeeST samples only from active nodes to generate walks or neighborhood subgraphs as training input. After training updates embeddings, FeeST computes the neighborhood-consistency metric to assess representation quality. Nodes that meet the consistency criterion are removed from the active set, while under-trained nodes remain active for the next round. By iteratively shrinking the active set, FeeST dynamically allocates sampling effort where it is still beneficial, reducing redundant exploration and speeding up convergence.

Feedback Mechanism. Graph embedding aims to preserve node attributes and graph structure in a low-dimensional space, such that nodes that are proximate or structurally related in the original graph remain similar in the embedding space. We therefore use embedding similarity with neighbors as a feedback signal to assess representation quality. Specifically, for a node v_i, if its embedding is sufficiently consistent with those of its neighbors, we treat v_i as well-trained and exclude it from future sampling rounds. We quantify this neighborhood-consistency as the average cosine similarity between v_i and its neighbors. Let $N(v_i)$ represent the set of neighbors of node v_i, and μ be a predefined similarity parameter. To mitigate the computational overhead of computing similarities for pair-wise nodes, FeeST adopts a neighbor subsampling strategy. Instead of evaluating similarities across the entire neighborhood, we estimate the neighborhood-consistency score $\Psi(v_i)$ using a sampled subset $\tilde{N}(v_i) \subseteq N(v_i)$:

$$\Psi(v_i) \approx \frac{1}{|\tilde{N}(v_i)|} \sum_{v_j \in \tilde{N}(v_i)} \frac{\vec{v_i} \cdot \vec{v_j}}{\|\vec{v_i}\| \|\vec{v_j}\|} \tag{4}$$

This approximation is effective because real-world graphs often exhibit redundant local neighborhoods. In many graphs, nodes with similar attributes or roles tend to connect, making neighboring nodes highly correlated [30, 42]; meanwhile, power-law degree distributions create hubs with many overlapping neighbors [5]. Empirical studies on GNNs also suggest that aggregating a small subset of neighbors is often sufficient, while using all neighbors yields diminishing returns but incurs extra overhead [12, 25].

An alternative is to detect convergence via self-stability, i.e., comparing a node's embedding across consecutive rounds. However, this requires storing historical embeddings, incurring additional $O(|V|{\cdot}d)$ memory overhead at scale. More importantly, graph embedding aims to preserve structural and attribute relationships with respect to the original graph. By evaluating consistency with neighbors (Eq. 4), FeeST aligns the feedback signal with this objective. In contrast, self-stability alone can yield false convergence: an embedding may change little across rounds yet remain inconsistent with its neighborhood, providing misleading guidance to the sampler. Finally, training-parameter signals such as gradient magnitudes are not reliable indicators of structural consistency, as they mainly capture within-iteration optimization dynamics and minibatch stochasticity [70, 86], and lack a cross-round structural view for updating the sampling frontier (i.e., the active set).

If $\Psi(v_i) < \mu$, the embedding of v_i is considered undertrained, and the node will be included in the active node set for the next round of sampling $V_a^{(r+1)}$. Otherwise, it is excluded from further sampling. The active node set for round $r+1$ is formally defined as:

$$V_a^{(r+1)} = \left\{ v_i \in V_a^{(r)} \,\middle|\, \Psi(v_i) < \mu \right\} \tag{5}$$

Empirically, we set the default μ to 0.65. With this feedback mechanism, FeeST treats active nodes as a round-level operational state rather than a static graph property or homophily-based label category: after each training round, a node remains active if $\Psi(v_i) < \mu$, indicating that its representation may still benefit from further sampling and training. This helps FeeST identify insufficiently stable embeddings, avoiding redundant sampling and computation without sacrificing downstream accuracy (§6). While Eq. 4 is most directly aligned with graphs where local similarity is informative, its scale becomes more dataset-dependent in heterophilic graphs because neighbors may not be semantically similar. Therefore, in heterophilic settings, we treat μ as a validation-calibrated decision boundary rather than a universally fixed threshold, optionally guided by structural homophily metrics [23, 48]. Specifically, we calibrate μ over a small candidate set using only the held-out validation split under the standard 50%/25%/25% split, select the best validation quality-efficiency trade-off, and use the test set only for final reporting. R2.E1.1 R3.D2

As shown in Figure 5(b), the number of active nodes decreases rapidly during the early rounds on the *YouTube* graph. This observation aligns with the power-law distribution commonly found in real-world graphs, where most nodes have a low degree. These low-degree nodes become sufficiently trained quickly due to sparse connectivity and limited information content, while high-degree nodes require more rounds of sampling to capture their neighborhood context. This validates that FeeST eliminates redundant exploration from the early rounds, unlike decoupled paradigms where all nodes continue to be sampled regardless of representation quality. Importantly, the active frontier is recomputed every round, enabling R3.D3

nodes to be reactivated when they become inconsistent with their neighborhoods. Inactive nodes may still be revisited through random walks or neighbor aggregation, allowing their embeddings to keep evolving. On the *ComOrkut* graph, we observe that 6.4% of nodes are reactivated per round on average. Moreover, Figure 5(c) shows that although FeeST starts pruning from the first round and rapidly shrinks the active frontier, its gradient variance remains comparable to w/o-FeeST in the first few rounds and becomes consistently lower afterward, suggesting that the induced non-stationary sampling does not introduce harmful optimization instability in practice.

Implementation. Algorithm 1 presents the workflow of FeeST, which iteratively performs sampling, training, and feedback until the active node set becomes empty. To ensure generalizability, FeeST abstracts both sampling and training through model-agnostic interfaces. The $\texttt{Sample}(v_i, \theta) \rightarrow W_{v_i}^L$ function generates either random-walk sequences (e.g., DeepWalk, node2vec, HuGE) or GNN-based neighborhood subgraphs with configurable fan-out, where v_i is the source node, θ denotes configurable parameters such as walk bias or neighbor budget, and $W_{v_i}^L$ is the resulting walk or subgraph. The $\texttt{Train}(W_{v_i}^L, model) \rightarrow \vec{V}$ function then consumes these samples to update embeddings using the corresponding model, such as Skip-Gram with negative sampling for random walks, or message passing and aggregation for GNNs. This unified abstraction allows FeeST to plug feedback into different embedding pipelines (details in §5.4).

Complexity Analysis. The computational complexity can be interpreted directly from the following phases: **Sampling (line 3).** Let V_a^r denote the active node set in round r, for each active node $v_i \in V_a^r$, the Sample function generates a walk or subgraph of average length L. The cost is $O(|V_a^r| \cdot L)$. **Training (line 4).** Embeddings are updated on the collected samples. With walk length L, window size w, and embedding dimension d, the training cost is $O(|V_a^r| \cdot L \cdot w \cdot d)$. **Feedback (lines 5-6).** For each active node, FeeST computes the neighborhood-consistency score $\Psi(v_i)$ using a subsampled neighborhood $\tilde{N}(v_i)$ of size $\bar{k}_s$, costing $O(|V_a^r| \cdot \bar{k}_s \cdot d)$. Since $|V_a^r|$ shrinks across rounds as sufficiently trained nodes are removed, the total runtime is $O\left(\sum_{r=1}^{R} |V_a^r| \cdot (L \cdot w \cdot d + \bar{k}_s \cdot d)\right)$, which can be substantially smaller than the decoupled baseline that repeatedly samples and trains over all nodes. The space complexity is dominated by embeddings $O(|V| \cdot d)$ at the current round, with temporary memory $O(\max_r |V_a^r| \cdot (L + \bar{k}_s))$ for sampled walks and neighbor subsets.

Algorithm 1 Feedback-driven Sampling-Training Coupling

Input: Graph $G = (V, E)$, embedding dimension d, parameter μ
Output: Node embeddings $\vec{V}$

1: Initialize active node set $V_a \leftarrow V$
2: **while** $V_a \neq \emptyset$ **do**
3: **Sampling:** For each $v_i \in V_a$, generate samples $W_{v_i}^L \leftarrow \texttt{Sample}(v_i, \theta)$
4: **Training:** Update embeddings $\vec{V} \leftarrow \texttt{Train}(W_{v_i}^L, Model)$
5: **Feedback:** For each $v_i \in V_a$, evaluate embedding quality $\Psi(v_i)$
6: Form next active set $V_a^{\text{next}} = \{v_i \in V_a \mid \Psi(v_i) < \mu\}$
7: $V_a \leftarrow V_a^{\text{next}}$
8: **return** $\vec{V}$

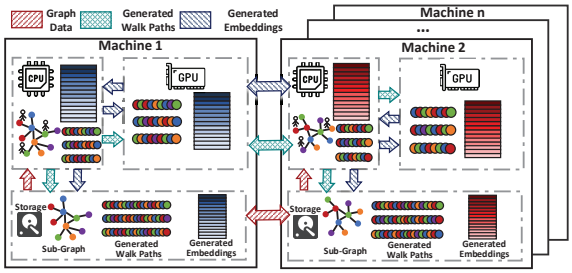


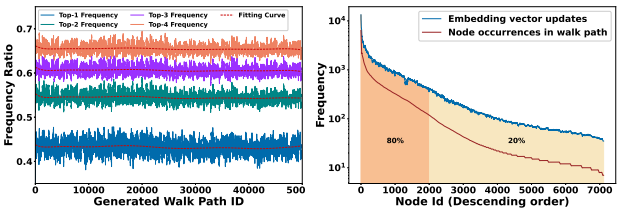
Figure 8: Data flow in the feedback-coupled sampling-training.

5.2 Activity-aware Communication Mechanism

In large-scale graph embedding systems, efficient communication is essential to overcome the bottlenecks introduced by both intra-machine sampling-training transmission and inter-machine embedding synchronization. As observed in § 4.2, communication overhead, particularly between the CPU and GPU, as well as across distributed machines, comprises a significant portion of the overall execution time (up to 78%), leading to low resource utilization and scalability issues. To address these challenges, we propose an *activity-aware communication mechanism* (ACM), which encompasses a *frequency-aware bitmap compression strategy* (FaBC) to alleviate PCIe bandwidth pressure and a *hotspot-aware synchronization strategy* (HaSyn) to reduce network traffic.

5.2.1 Analysis of Data Flow in FeeST. As shown in Figure 8, the communication process can be summarized in the following stages: (1) ***Graph data loading and distribution:*** Raw graph data is read from storage into CPU memory and partitioned across worker nodes (red arrows). (2) ***Sampling and path generation:*** Each worker performs random walks on its local subgraph, and path fragments are exchanged if walks cross partitions. The generated walk paths are then aggregated for training (green arrows). (3) ***CPU-GPU transfer for training:*** Aggregated sequences are transmitted from CPU to GPU memory as training input. This PCIe transfer grows with the volume of samples and often emerges as a major bottleneck (green arrows). (4) ***Distributed synchronization:*** During training, updated gradients or embeddings are copied from GPU to CPU, aggregated across machines, and written back to GPUs for parameter updates. This synchronization dominates communication costs in distributed environments (blue arrows). (5) ***Persistence:*** After convergence, final embeddings are transferred back to CPU memory and persisted to storage (blue arrows). In addition, generated walk paths are periodically checkpointed for reproducibility and recovery purposes (green arrows). Among these stages, (3) CPU–GPU transfer and (4) distributed synchronization are the two dominant bottlenecks during communication, which motivate the design of FaBC and HaSyn.

5.2.2 Frequency-aware Bitmap Compression Strategy. In graph embedding systems, sampling generate node sequences or subgraphs as training input. Due to the skewed degree distribution in real-world graphs, high-degree nodes tend to be revisited more frequently, and random walks are more likely to repeatedly traverse their local neighborhoods [20, 35]. Existing strategies (e.g., biased transitions in node2vec or backtracking in HuGE) exacerbate this issue by favoring repeated exploration of local neighborhoods. As a result, sampled paths or subgraphs contain many duplicate node occurrences



(a) Node Occurrences (b) Embedding Updates

Figure 9: Distribution characteristics of node occurrences and embedding updates.

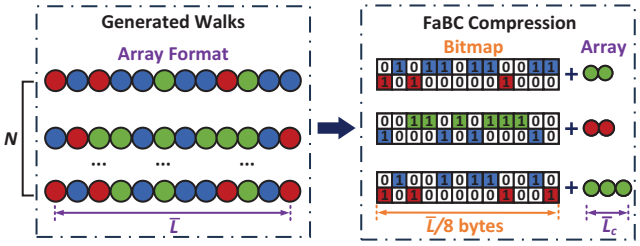


Figure 10: Frequency-aware compression process.

that must be transferred from CPU to GPU for training, imposing substantial PCIe bandwidth overhead. However, existing systems typically materialize samples as plain arrays of node IDs, which fails to exploit this repetition. Since redundancy is dominated by a small set of frequent nodes, encoding their occurrence patterns with compact bitmap representations [11] is often far more efficient than repeatedly sending the same identifiers.

We propose the *Frequency-aware Bitmap Compression* (FaBC) strategy, which identifies high-frequency nodes and records their positions as compact bitmaps, turning repeated identities into lightweight markers for CPU-GPU transfer. Unlike prior bitmap techniques that mainly target static bitvectors or graph adjacencies, FaBC is workload-driven and tailored to the highly repetitive access patterns in sampling-generated walk paths, and is co-designed with FeLoG's workflow to reduce the dominant transfer bottleneck. While finer-grained frequency buckets can improve compression, they also introduce extra bitmap construction and decoding overhead. FaBC therefore selects a small number of buckets (or ranges) based on the observed node-frequency distribution to balance compression effectiveness and runtime cost.

To this end, FaBC determines the compression range by profiling the node-frequency distribution in the first sampling–training round. After the initial sampling, we compute normalized node frequencies on the sampled sequences and sort them as $p(1) \geq p(2) \geq \cdots$. We define the cumulative frequency of the top-k nodes as $F_k = \sum_{i=1}^{k} p(i)$. Under FaBC, selecting the top-k nodes compresses their repeated occurrences into per-sequence bitmaps and stores the remaining (infrequent) nodes in the original ID form. For N sequences with average length L, the baseline storage is $S_0 = 4NL$ bytes (4 bytes per node ID). After compressing the top-k nodes, the uncompressed tail accounts for a $(1-F_k)$ fraction of positions and costs $4NL(1-F_k)$. We store the k selected node IDs once per sequence, costing $4Nk$, and store one length-L bitmap per selected node per sequence, costing $Nk \cdot (L/8)$ bytes. Thus, the compressed size is

$$S(k) = 4NL(1 - F_k) + 4Nk + Nk\frac{L}{8} \tag{6}$$

and the compression ratio is

$$g(k) = \frac{S(k)}{S_0} = 1 - F_k + \frac{k}{L} + \frac{k}{32} \tag{7}$$

Here, $1 - F_k$ is the fraction of positions that remain uncompressed, $\frac{k}{L}$ corresponds to storing the k selected IDs once per sequence, and $\frac{k}{32}$ is the bitmap overhead (1 bit per position).

FaBC expands the compression target set only when the $(k+1)$-th node provides sufficient marginal benefit. Let $c = \frac{1}{L} + \frac{1}{32}$ denote the amortized overhead of adding one more compressed node, so $g(k) = 1 - F_k + kc$. Since $F_{k+1} = F_k + p(k+1)$, we have $g(k+1) = g(k) - p(k+1) + c$, To avoid diminishing returns, FaBC selects the largest k satisfying

$$\frac{g(k) - g(k+1)}{g(k)} \geq 0.1 \quad \Longleftrightarrow \quad p(k+1) \geq c + 0.1g(k) \tag{8}$$

which requires the marginal reduction to be at least 10% of the current ratio. The selected top-k nodes are then used as compression targets, and their occurrences are encoded with bitmaps in subsequent samples. In practice, the frequency profile is largely stable under fixed sampling parameters, so the configuration can be reused without repeated analysis; if needed, FaBC can refresh the profile periodically with low overhead.

Example. Figure 9(a) illustrates the node frequency curve of walks generated by FeeST on the *YouTube* graph. The top-1 and top-2 nodes together contribute over 50% of the total frequency. Including the third most frequent node increases the cumulative frequency by less than 10%, and adding the fourth brings only marginal improvement. Hence, according to the above criterion, FaBC selects the top-1 and top-2 nodes as compression targets. This demonstrates how the adaptive rule captures dominant redundancy without extra bitmaps. The impact of different frequency ranges on compression ratio is further evaluated in Table 6.

As illustrated in Figure 10, FaBC achieves notable compression. In a set of N random walks with average length $\overline{L}$, storing nodes as 4-byte unsigned integers results in a raw size of $4N\overline{L}$ bytes. After compression, the positions of the top-k frequent nodes are stored using k bitmaps of length N, while the remaining nodes are stored as an ID stream. Let $\overline{L_c}$ denote the average length of the remaining sequence. From Figure 9(a), when the top-2 nodes cover approximately 60% of the walk content, we have $\overline{L_c} \approx 0.4\overline{L}$. Therefore, the compressed data size is approximately $\frac{k \cdot N\overline{L}}{8} + 4N\overline{L_c} \approx 0.25N\overline{L} + 1.6N\overline{L}$ bytes, which significantly reduces the training data size while incurring minimal computational overhead (details in 6.6).

5.2.3 Hotspot-aware Synchronization Strategy. Synchronization of embedding vectors is a critical factor influencing both accuracy and system performance in distributed training. Existing methods such as parameter-server based designs (e.g., PBG [33] and NuPS [51]) and optimized communication topologies (e.g., TidalMesh [38]) improve efficiency, but they typically treat all embeddings equally, ignoring the skewed update activity across nodes. In real-world graphs, the sampling workload is highly skewed, and a small set of high-frequency nodes appears far more often in sampled sequences, leading to disproportionately more embedding updates. As shown

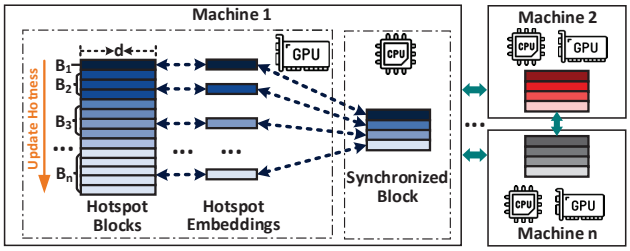


Figure 11: Hotspot-aware Synchronization Strategy across distributed machines.

R1.D1 in Figure 9(b), more than 80% of updates on the *Wiki-Vote* graph [34] are concentrated on a small set of high-frequency nodes, while the majority of embeddings are rarely updated. This makes uniform full-model synchronization inefficient, as most bandwidth is wasted transmitting nearly static vectors, motivating hotness-aware selective synchronization that prioritizes frequently updated embedding.

R1.W1 To address this issue, we propose the *Hotspot-aware Synchronization Strategy* (HaSyn), which dynamically adjusts the synchronization frequency of embedding vectors based on their update hotness. As illustrated in Figure 11, during the initialization phase, the embedding matrix is constructed by sorting nodes in descending order of their frequencies observed in the training data, where the frequency statistics are collected asynchronously during sampling by a background thread. During synchronization, we partition $\vec{V}$ into hotspot blocks $B_f = \{v \mid f(v) = f\}$ indexed by distinct frequency values $f \in \mathcal{F}$, yielding $|\mathcal{F}|$ blocks in total, and randomly select one node from each block B_f to form a synchronization set of size $|\mathcal{F}|$. Due to the skewed frequency distribution, a node in block B_f is selected with probability $1/|B_f|$, making hot embeddings (typically in smaller blocks) more likely to be synchronized, while cold embeddings are synchronized less frequently. Thus, HaSyn avoids repeated global graph analysis or full re-ranking in each synchronization period, and maintaining the synchronization set only requires lightweight block-level lookup/update operations. Compared to full-model synchronization, the communication cost of HaSyn is reduced to $O(|\mathcal{F}| \cdot d \cdot M)$, where d is the embedding dimension and M is the number of machines, with $|\mathcal{F}| \ll |V|$ in practice (details in §6.6).

R2.E1.3 Furthermore, under FeeST (§5.1), the active frontier shrinks across rounds, so update activity concentrates on a progressively smaller subset of embeddings and shifts over time. HaSyn therefore prioritizes embeddings with high update frequency and adapts to cross-round workload shifts, reducing *inter-machine* synchronization volume while avoiding indefinite staleness through periodic synchronization of cold embeddings. As a lightweight hotspot-based strategy, HaSyn is most effective when update activity is skewed; under nearly uniform updates or stricter consistency requirements, more sophisticated graph partitioning and adaptive synchronization techniques can be complementary to FeLoG. We analyze the resulting quality-efficiency trade-off of selective synchronization in §6.6.

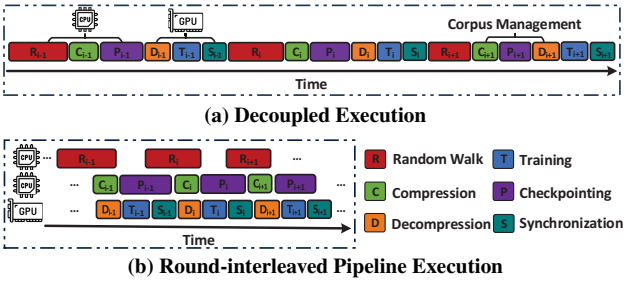
5.3 Round-interleaved Pipeline Parallelism

While FeLoG improves end-to-end efficiency via the FeeST model (§5.1) and the ACM mechanism (§5.2), its execution still remains *round-synchronous* and *decoupled* between sampling and training. As discussed in §4.2, training starts only after sampling for the current round is fully completed and materialized, which limits CPU-GPU concurrency and leads to low utilization leading to a low average utilization ratio (less than 26%).

Execution Analysis of FeLoG Components. Figure 12(a) depicts the decoupled execution mode of FeLoG, where in each round the training phase is launched only after the sampling phase finishes completely. Furthermore, there exists an intermediate phase, *corpus management*, between sampling and training. This includes: (1) the compression and decompression of walk paths using FaBC, as introduced in §5.2.2, and (2) checkpointing the sampled data to persistent storage for fault tolerance. Specifically, after each round of sampling, the system performs compression on the CPU. This step significantly reduces the size of the walk paths by identifying and encoding high-frequency nodes via compact bitmap indices, thereby lowering the PCIe transmission load from CPU to GPU. The compressed walk sequences are then serialized and checkpointed to persistent storage to ensure fault tolerance. Later, during training, the decompression is executed on the GPU side. The GPU first transfers the compressed sequences into its memory, then reconstructs the original node sequences by decoding the bitmap and reassembling the full walk paths. Afterwards, the updated embeddings are synchronized across machines by the hotspot-aware strategy (§5.2.3). Because sampling and training are separated by round-level barriers and corpus materialization, both CPU and GPU can be idle around these intermediate steps, resulting in poor overall resource utilization.

Pipeline Execution Design. To overcome the limitations of decoupled execution, FeLoG introduces a *round-interleaved pipeline parallelism* (RiPP) to fully exploit the parallelism of heterogeneous systems and reduce idle time. The core idea of RiPP is to analyze the data dependencies across the sampling and training phases, decouple their operators into fine-grained execution units, and schedule them in an interleaved, round-wise pipeline to maximize concurrency. As illustrated in Figure 12(b), each sampling-training round in FeLoG is decomposed into six operators: *random walk generation* (R), *data compression* (C), *checkpointing* (P), *data decompression* (D), *model training* (T), and *synchronization* (S). For instance, R_i denotes the random walk generation operator in round i, and T_i corresponds to the associated training operator that consumes the walk corpus. Unlike DistGER-Pipe [18], which overlaps sampling and training in a decoupled CPU-only pipeline driven by pre-defined graph-structural signals, RiPP targets feedback-coupled CPU-GPU execution in FeLoG. Specifically, the sampling frontier of round $i+1$ depends on the embedding-quality feedback produced after training round i, so R_{i+1} must preserve this one-round feedback dependency rather than being launched independently of T_i. This feedback dependency also differentiates RiPP from general GNN pipeline schedulers such as NeutronOrch [2], DAHA [36], and LeapGNN [13], which mainly optimize execution scheduling without modeling FeeST's round-level embedding-quality feedback. R2.E1.4

To maximize concurrency under this feedback dependency, RiPP adopts a three-group operator scheduling strategy. It groups and dispatches: ❶ CPU-side random walk generation (R), ❷ CPU-side data compression (C) and checkpointing (P), and ❸ GPU-side data decompression (D), model training (T), and synchronization (S) into three asynchronous operator pipelines, each mapped to the most



(a) Decoupled Execution

(b) Round-interleaved Pipeline Execution

Figure 12: Execution flow comparison in FeLoG.

appropriate hardware resources. These operators form a hybrid parallelism scheme: *intra-round* serial execution for dependent operators on the same data chunk, and *inter-round* parallel execution across different threads and devices whenever their dependencies are satisfied. For example, once walk chunks of round i are generated on the CPU, C_i and P_i proceed on CPU-side threads, while ready chunks from adjacent rounds are decompressed, trained, and synchronized on the GPU. Under RiPP, active-frontier construction is separated from random-walk generation. After T_{i-1}, each machine computes $\Psi(v)$ for its local candidate nodes using the locally visible embedding snapshot, which contains local training updates and previously synchronized remote embeddings. Each machine then materializes its local portion of $V_a^{(i)}$, allowing R_i to start without waiting for the current synchronization S_{i-1} to complete. This use of locally visible or slightly stale embeddings follows common distributed graph learning practice [4, 61, 85]. Once $V_a^{(i)}$ is fixed, R_i no longer computes $\Psi(v)$ or accesses embeddings, but only performs graph-structure-based walk generation from active nodes. Thus, R_i can overlap with S_{i-1}, which asynchronously propagates previous-round embedding updates for subsequent training and later active-frontier construction. This design keeps CPU and GPU resources continuously utilized while preserving the feedback dependency required by FeeST, thereby improving end-to-end training efficiency.

R2.E1.2

5.4 Integration of FeLoG with Existing Systems

Modern graph embedding frameworks can be broadly classified into two categories: random walk-based systems (e.g., DeepWalk [47], node2vec [24], HuGE [21], DistGER [17]) and GNN-based systems (e.g., GraphSAGE [25], DistDGL [85], NeutronTP [3]). FeLoG is designed as a lightweight optimization layer that can be seamlessly incorporated into both paradigms through two unified interfaces, `Sample` and `Train`. This abstraction decouples state-of-the-art embedding models from our proposed feedback loop, enabling system-level integration without modifying the core model design.

Sample Interface. The sampling process is abstracted as $\texttt{Sample}(v_i, \theta) \rightarrow W_{v_i}^L$, where v_i is the source node, θ denotes model-specific parameters (e.g., bias factor, neighbor budget, entropy criterion), and $W_{v_i}^L$ is either a node sequence or a neighborhood subgraph of length L. This unified form supports: (1) *Random-walk systems*, where `Sample` generates walks such as uniform walks in DeepWalk, second-order walks in node2vec, or biased walks toward informative neighbors in HuGE/DistGER; (2) *GNN systems*, where `Sample` extracts subgraphs with configurable fan-out for message passing. Within FeLoG, the `Sample` stage is enhanced by two mechanisms: *FeeST* dynamically updates the active node sets to avoid redundant walks, and *FaBC* compresses generated sequences to reduce CPU-GPU transmission.

Train Interface. The training process is abstracted as $\texttt{Train}(W_{v_i}^L, model, feedback = True) \rightarrow \vec{V}$, where $W_{v_i}^L$ denotes the sampled corpus, *model* is the embedding architecture, and $\vec{V}$ is the resulting embedding matrix. For random walk systems, this corresponds to Skip-Gram with negative sampling (Eq. 1, Eq. 2), while for GNN systems it corresponds to iterative message passing and aggregation (Eq. 3). FeLoG integrates feedback directly into this step: after each update, *FeeST* evaluates embedding quality and prunes converged nodes, while *HaSyn* adaptively synchronizes high-frequency embeddings across machines to reduce network costs. In GNNs, *FaBC* can be reused to compact redundant neighbor lists, and *HaSyn* applies to hotspot embedding vectors arising from skewed node access.

Unified Deployment View. From a system perspective, the deployment of FeLoG can be summarized as:

$$\texttt{S = Sample}(G, \theta) \quad \text{(FeeST + FaBC)}$$

$$\vec{V} \texttt{ = Train}(S, model, feedback = True) \quad \text{(FeeST + HaSyn)}$$

$$\texttt{Persist}(S, \vec{V})$$

$$\texttt{RiPP}(\{Sample, Train, Persist\}) \quad \text{(Pipeline execution)}$$

This API-style abstraction emphasizes that FeLoG is not a standalone framework, but a modular optimization layer that can be embedded into both CPU-GPU and multi-machine distributed environments. In random-walk frameworks such as DistGER, all three mechanisms (FeeST, FaBC, HaSyn) are integrated together with pipeline execution (RiPP). In GNN frameworks, FeLoG mainly leverages (FeeST) and can extend FaBC and HaSyn to reduce neighbor redundancy and hotspot synchronization. Our experiments in §6 demonstrate the end-to-end benefits of this integration when applied to DistGER, while also validating its extensibility to GNN-based systems such as DistDGL and NeutronTP.

6 EXPERIMENTAL RESULTS

We evaluate the efficiency (§6.2) and scalability (§6.3) of our proposed method, FeLoG by comparing with PyTorch-BigGraph (PBG) [33], Distributed DGL (DistDGL) [85], DistGER [17], DistGER-Pipe [18], LeapGNN [13], and NeutronTP [3]. We also compare the effectiveness (§6.4) of generated embeddings on link prediction, graph-based RAG, and node classification. Finally, we analyze the generalizability of FeLoG for the GNN-based embeddings (§6.5) and efficiency due to individual parts of FeLoG (§6.6). **Our codes and datasets are at [1].**

6.1 Experimental Setup

Environment. We conduct experiments on a cluster of 8 machines, each equipped with a 2.60GHz Intel® Xeon® Gold 6240 CPU (36 cores with 72 threads), a 32GB NVIDIA V100 GPU, 192GB DDR4 memory, and interconnected via full-duplex 10 Gbps networking. The machines run Ubuntu 20.04, with GCC 9.4.0 used for compiling DistGER, DistGER-GPU, and FeLoG, and Python 3.10 with torch 1.11.0 used as the backend deep learning framework for PyTorch-BigGraph, DistDGL, LeapGNN, NeutronTP, and GraNNDis. The multi-GPU comparison with GraNNDis is conducted on a server equipped with four NVIDIA A40 GPUs.

R1.W2

R2.D3

Table 3: Datasets statistics (K=10^3, M=10^6, B=10^9) and Avg. memory footprint (GB) per machine. "✗" indicates failure due to out of memory or > 1 day runtime.

Graph	Datasets statistics		Avg. memory footprint per machine (GB, CPU/GPU)						
	#Nodes	#Edges	PBG	DistDGL	DistGER	DistGER-G	NeutronTP	LeapGNN	FeLoG
FL	80.51 K	5.90 M	8.9/-	5.4/1.4	0.9/-	0.5/0.9	8.6/1.2	6.8/1.2	0.5/0.5
YT	1.14 M	2.99 M	9.7/-	5.5/1.5	4.3/-	2.3/5.3	9.0/1.6	17.0/3.4	1.9/0.6
LJ	2.24 M	14.61 M	10.3/-	5.8/1.7	5.5/-	2.6/6.6	9.3/2.2	30.7/7.7	2.1/0.9
OR	3.07 M	117.19 M	11.4/-	6.3/2.1	6.9/-	6.0/8.0	9.7/2.6	39.4/20.1	6.1/1.8
TW	41.65 M	1.47 B	31.7/-	✗	67.2/-	✗	19.6/19.4	✗	59.7/7.2
OGB	111.06 M	1.62 B	✗	✗	72.3/-	✗	30.7/31.2	✗	71.5/9.4
UK	105.15 M	3.72 B	✗	✗	125.8/-	✗	✗	✗	107.3/9.0

Datasets. We employ seven widely-used, real-world graphs (Table 3): *Flickr* (FL) [56], *Youtube* (YT) [56], *LiveJournal* (LJ) [78], *Com-Orkut* (OR) [74], *Twitter* (TW) [31], *U.K.-2007* (UK) [6] and *OGB-papers100M* (OGB) [27]. The *FL* and *YT* datasets are labeled, with *FL* having 195 labels for user-content interactions and *YT* 47 labels for video engagement. The *OGB* dataset has 128-dimensional node features, representing citation networks of scientific papers. For heterophilic settings, we use two labeled benchmarks, *Amazon-ratings* (AR: $|V|$ = 24.49K, $|E|$ = 93.05K, 5 classes) and *Roman-empire* (RE: $|V|$ = 22.66K, $|E|$ = 32.93K, 18 classes) [49], to evaluate the sensitivity of FeeST to μ via downstream node classification. R2.E1.1 We also use synthetic graphs [10] (up to 1 billion nodes, 10 billion edges) to assess the scalability of FeLoG. Considering the default settings of popular graph embedding methods, we use their undirected version.

Baselines. We select six state-of-the-art distributed graph embedding frameworks: the multi-relation embedding system PyTorch-BigGraph (PBG) developed by Facebook https://github.com/facebookresearch/PyTorch-BigGraph [33]; the GNN-based system DistDGL from Amazon https://github.com/dmlc/dgl [85]; the recent GNN-based framework NeutronTP https://github.com/iDC-NEU/NeutronTP [3] and LeapGNN https://github.com/ISCS-ZJU/LeapGNN-AE [13]; and the random walk-based system DistGER https://github.com/RocmFang/DistGER [17]. In addition, we include DistGER-Pipe [18], a CPU-only pipeline-optimized extension of DistGER, and further implement its hybrid CPU-GPU variant, DistGER-G, which performs sampling on CPUs and offloads training to GPUs. We additionally compare with GraNNDis https://github.com/AIS-SNU/GraNNDis_Artifact [54], a recent multi-GPU GNN training framework. R1.W2 We do not compare against single-machine execution planners such as NeutronOrch [2], DAHA [36], and GIDS [45], as they target intra-node scheduling and are not designed for FeLoG's multi-machine setting. Empirically, in our setup on *OR*, SSD-based GIDS takes 3095 s while FeLoG completes in 61.8 s, and the in-memory NeutronOrch/DAHA run out of memory on the billion-scale *UK* graph.

Parameters. By default, FeLoG uses random-walk based graph embedding, while we also evaluate its generality over GNN-based embeddings in §6.5. We set μ=0.65 in FeeST, and compress the top-2 most frequent nodes in FaBC (the sensitivity analysis is reported in § 6.7). For DistGER and DistGER-G, we set the sliding window size w=10, the number of negative samples K=5, and the synchronization period to 0.1 sec [28]. For PBG, DistDGL, LeapGNN and NeutronTP, we follow the default settings in their papers [3, 13, 33, 85]. For fair system comparison, we use embedding dimension d=128 that is commonly adopted [21, 24, 47, 57, 79], and report the average end-to-end epoch time. For task effectiveness, we select the best results via grid search over learning rates in $[0.001, 0.1]$, training epochs in $[1, 30]$, embedding dimensions in $\{32, 64, 128, 256, 512\}$, and, for neighbor-sampling based GNN settings, sampling hyper-parameters including the number of layers $\in \{2, 3, 4\}$ and per-layer fanout $\in \{5, 10, 15, 25\}$.

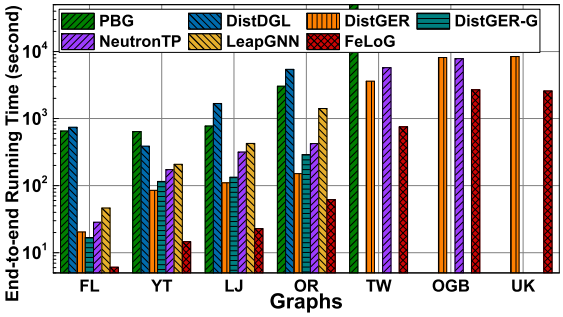
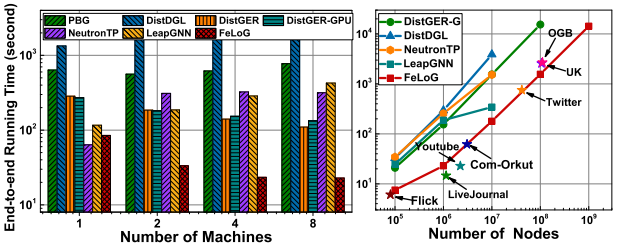


Figure 13: Efficiency of PBG **[33],** DistDGL **[85],** DistGER **[17],** DistGER-G **[18],** NeutronTP **[3],** LeapGNN **[13] and** FeLoG **(ours).**

6.2 Efficiency and Memory Requirement

We report the end-to-end running times of PBG, DistDGL, DistGER, DistGER-G, NeutronTP, LeapGNN, and FeLoG on seven real-world graphs with the cluster of 8 machines in Figure 13. The reported average epoch end-to-end time includes sampling, training, and communication overheads. FeLoG significantly outperforms the competitors on all these graphs, achieving an average acceleration of 27.9×. PBG leverages a parameter server to synchronize embeddings between clients, resulting in more load on the communication network. As a result, PBG is on average 68.4× slower than FeLoG. As a system of the same type, FeLoG outperforms the CPU-based DistGER and the CPU-GPU hybrid DistGER-G by 3.9× and 5.3× on average, respectively. DistGER uses information-theoretic walks to adapt sampling, but without training feedback it applies a uniform configuration to all nodes, causing coarse-grained control and redundancy. DistGER-G, though accelerating training via GPU, incurs heavy CPU-GPU data transfer overhead, which reduces GPU utilization and degrades overall performance. Notice that DistGER-G fails on larger datasets such as *TWT*, *OGB*, and *UK* due to its redundant corpus generation during sampling and suboptimal system support for training, which lead to memory consumption exceeding available capacity. For GNN-based system DistDGL, due to the long running time of graph sampling (e.g., taking 80% of the overhead), it is highly inefficient than other systems. Similarly to DistGER-G, it cannot run on billion-edge graphs, as it fails to terminate within one day. Two recent GNN systems, NeutronTP and LeapGNN, optimize communication and feature operations. However, their lack of real-time embedding quality awareness and uniform treatment of node features across iterations lead to redundant computation and hinder convergence speed. Hence, they are on average 7.9× and 15.8× slower than FeLoG, and cannot run on the largest dataset *UK* due to out-of-memory issues. We further discuss accuracy-time convergence in §6.4 to quantify the time-to-quality tradeoff. R2.E2.2

Considering the resource consumption that affects scalability, Table 3 reports the average per-machine memory footprint of FeLoG and baseline systems on each graph. FeLoG consistently achieves the lowest or one of lowest memory usage across both CPU and



(a) Scalability across Machine Scales (b) Scalability across Graph Sizes

Figure 14: Scalability analysis of FeLoG.

GPU components, with particularly low usage on GPU. For the billion-scale graphs like *UK*, FeLoG is one of the few systems able to run without exceeding memory limits, highlighting its efficiency and scalability in distributed settings.

6.3 Scalability

Figure 14(a) shows end-to-end running times of all competing systems on the *LiveJournal* graph, as we increase # machines from 1 to 8 to evaluate scalability. FeLoG achieves better scalability than the other six distributed systems. Due to space limitations, we omit results on other graph datasets, which either exhibit similar trends or are too large for the competing systems to scale. PBG leverages a parameter server and a shared network filesystem to synchronize the parameters in the distributed model. When the number of machines increases, PBG puts more load on the communications network, resulting in poor scalability. DistDGL suffers from limited scalability due to its full-graph parameter synchronization and frequent gradient communication across machines. The CPU-based DistGER exhibits better scalability than the CPU-GPU hybrid DistGER-G, as the latter suffers from substantial CPU-GPU data transfer overhead and a sequential execution pattern that undermines the performance benefits of GPU acceleration. NeutronTP's end-to-end running time remains nearly stable as the number of machines increases, indicating limited scalability. This is mainly due to its uniform processing of all node features, leading to redundant computation and underutilization of distributed resources. LeapGNN scales poorly due to frequent step-wise global synchronization that amplifies stragglers and idle GPUs as machines increase. In comparison, FeLoG incorporates a feedback-driven sampling-training coupling model and an activity-aware communication mechanism to reduce both computation and communication costs, it also employs a round-interleaved pipeline to maximize CPU-GPU utilization. Hence, FeLoG achieves better scalability than other systems.

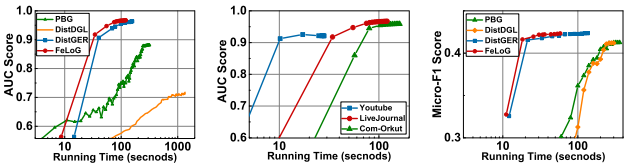
To further assess the scalability of FeLoG, we generate synthetic graphs [10] with a fixed node degree of 10 and the number of nodes from 10^5 to 10^9. Figure 14(b) presents the running times on these synthetic graphs using a cluster of 8 machines, suggesting that the running time increases linearly with the size of a graph, and FeLoG has the capability to handle even billion-node graphs. Moreover, the running times for seven real-world graphs (including the billions-scale graphs, for which the competing systems do not terminate in 1 day or crash due to memory limitation) are inserted into the plot, which is consistent with the trend on synthetic data.

Table 4: *AUC* of FeLoG and competitors for link prediction.

Method	Youtube	LiveJournal	Com-Orkut	Twitter
PBG	0.753	0.882	0.955	0.912
DistDGL	0.894	0.718	0.815	running time > 1 day
DistGER	**0.966**	0.976	0.921	0.919
FeLoG	0.949	**0.979**	**0.965**	**0.942**

Table 5: RAG retrieval performance of FeLoG and NeutronTP on *Flickr* and *Youtube* datasets (K=20, 2-hop retrieval).

Metric	Flickr		Youtube	
	NeutronTP	FeLoG	NeutronTP	FeLoG
Hit@K	0.817	**0.955** (+16.9%)	0.790	**0.860** (+8.9%)
nDCG@K	0.202	**0.499** (+147.0%)	0.267	**0.677** (+153.6%)
MRR@K	0.343	**0.714** (+108.2%)	0.430	**0.924** (+114.9%)



(a) Time-AUC **(b) FeLoG Across Graphs** **(c) Time-Micro-F1**

Figure 15: The influence of running time on embedding quality for FeLoG and competitors. Each point corresponds to one training epoch.

6.4 Effectiveness

Application in Link Prediction. To perform link prediction on a given graph G, following [17, 57, 75], we first uniformly at random remove 50% edges as positive test edges, and the rest are used as positive training edges. We also provide negative training and test edges by considering those node pairs between which no edge exists in G. We ensure that the positive and negative set sizes are similar. For a pair of nodes (u, v), let $\varphi(u)$ and $\varphi(v)$ be the vectors learned by embedding methods. The link prediction is conducted as a classification task based on the similarity of u and v, i.e., $\varphi(u) \cdot \varphi(v)$. The effectiveness of link prediction is measured via the *AUC* (Area Under Curve) score [76], the higher the better. We repeat this procedure 50 times to offset the randomness of edge removal and report the average *AUC* in Table 4. FeLoG outperforms all competitors on these graphs, except for DistGER on *Youtube*, where FeLoG ranks second. On average, FeLoG has an 9.8% higher *AUC* score compared with the other three systems. Figure 15(a) exhibits accuracy-efficiency tradeoffs of FeLoG and competitors, i.e., their *AUC* score convergence curves w.r.t. increasing running times, over *LiveJournal*, indicating that FeLoG tends to reach its peak accuracy in fewer epochs (around 4). In addition, Figure 15(b) shows consistently fast accuracy-efficiency convergence of FeLoG across graphs, reaching high *AUC* within a short running time on *Youtube* (3 epochs) and *Com-Orkut* (6 epochs). R2.E2.2

Application in Multi-label Classification. This task predicts one or more labels for each graph node and has applications in text categorization [81] and bioinformatics [80]. We use embedding vectors and

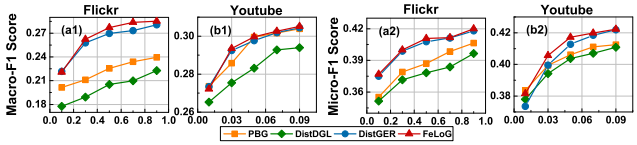
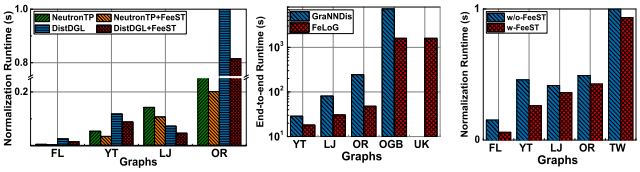


Figure 16: *Macro* − *F*1 **(a1, b1) and** *Micro* − *F*1 **(a2, b2) scores for multilabel node classification.**



(a) Generalizability **(b) Multi-GPU** **(c) FeeST Efficiency**

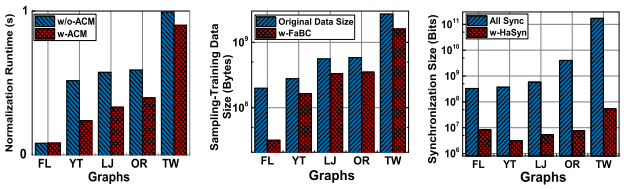
Figure 17: FeLoG **generalizability and** FeeST **impact on efficiency.**

a one-vs-rest logistic regression classifier with L2 regularization [16], then evaluate the effectiveness by micro-averaged F1 (*Micro* − *F*1) and macro-averaged F1 (*Macro* − *F*1) [62] scores, where *Micro* − *F*1 gives equal weight to each test instance and *Macro* − *F*1 assigns equal weight to each label category [29]. Following [21, 24, 47, 57], we select 10% to 90% training data ratio on *Flickr*, and 1% to 9% training ratio on *Youtube*. We report the averaged *Macro* − *F*1 and *Micro* − *F*1 scores in Figure 16. We find that FeLoG has better *Macro* − *F*1 and *Micro* − *F*1 scores than existing frameworks, gaining 9.8% and 2.8% average improvements, respectively, due to its more effective information-centric random walks. Figures 15 (c) show the time-accuracy trade-offs of FeLoG and competitors on node classification over *YTB*. The results also show that FeLoG reaches the same or higher accuracy in less time. R2.E2.2

Application in Graph-based RAG. To evaluate the generalization of FeLoG to graph-based retrieval-augmented generation (GraphRAG), where graph embeddings serve as the retrieval backbone for LLMs, we integrate the learned embeddings into a FAISS-based vector retrieval pipeline on *FK* and *YTB*. *FK* captures user-content interactions, while *YTB* models large-scale user-item engagement, enabling us to construct ground-truth relevance based on shared semantic attributes reflected by these interactions. Given a query node, we retrieve top-*K* candidate nodes by embedding similarity and evaluate retrieval quality using *Hit@K*, *nDCG@K*, and *MRR@K* (Table 5). Across both datasets, FeLoG consistently outperforms NeutronTP, achieving an average improvement of 150.3% in *nDCG@20*, 12.9% in *Hit@20*, and 111.6% in *MRR@20*. These results indicate that FeLoG yields higher-quality retrieval embeddings for GraphRAG.

6.5 Generalizability of FeLoG

A key feature of FeLoG is its general API design, which supports seamless integration with both random walk-based and GNN-based systems (§5.1). To validate this generalizability, we extend two representative distributed GNN systems, NeutronTP and DistDGL, by incorporating the feedback-coupled sampling-training (FeeST) via the unified API of FeLoG, resulting in NeutronTP+FeeST and DistDGL+FeeST. As shown in Figure 17(a), FeLoG consistently reduces normalization runtime across datasets. For NeutronTP, it achieves speedups between 1.33× and 1.69×, with an average of 1.50×. For



(a) ACM Efficiency **(b) Training Reduction** **(c) Sync Reduction**

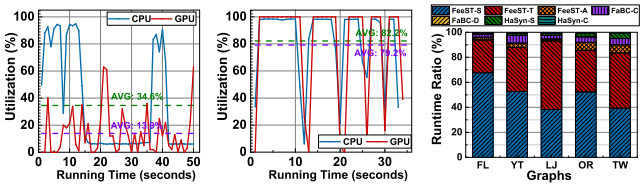
Figure 18: Impact of ACM **on communication performance.**

DistDGL, the speedups range from 1.23× to 1.67×, averaging 1.45×. These results demonstrate that the proposed feedback loop mechanism generalizes beyond random walk-based solutions and can be effectively applied to GNN-based pipelines, yielding significant efficiency gains without altering their underlying training logic. We further compare FeLoG with GraNNDis [54], a recent multi-GPU GNN training framework, under a four-NVIDIA-A40 setting. As shown in Figure 17(b), FeLoG achieves an average 3.45× speedup over GraNNDis across datasets, further confirming its ability to support and accelerate multi-GPU environment. R1.W2 R3.D3

6.6 Performance due to Individual Parts of FeLoG

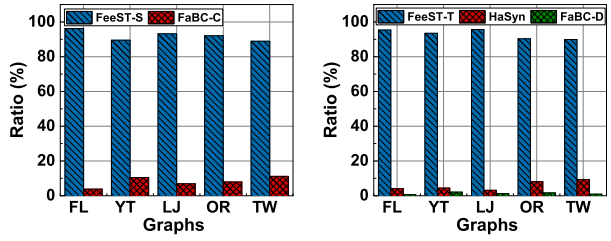
Feedback-driven Sampling-Training Efficiency. To evaluate the effectiveness of FeLoG's design (§5), we first examine the impact of the feedback-driven sampling-training coupling mechanism (FeeST, §5.1). We compare the execution time of FeLoG with and without FeeST (denoted as w-FeeST and w/o-FeeST) across all graphs. In w/o-FeeST, the number of sampling iterations is determined using the same strategy as DistGER, while all other settings remain consistent. As shown in Figure 17(c), w-FeeST achieves significant acceleration, ranging from 1.6× to 5.5×. This improvement comes from the dynamic feedback loop between sampling and training, where sampling is adaptively guided by real-time embedding quality indicators, effectively reducing redundant sampling and unnecessary computation. We further assess the sensitivity of FeeST to the similarity parameter μ in §6.7.

Communication Efficiency. We next evaluate the impact of the activity-aware communication mechanism (ACM, §5.2) on FeLoG's communication performance in distributed settings. Specifically, we compare the end-to-end runtime of FeLoG with and without ACM, denoted as w-ACM and w/o-ACM, respectively. As shown in Figure 18(a), enabling ACM yields an average speedup of 6.19× across all datasets. This substantial gain comes from two complementary strategies in ACM: FaBC (§5.2.2) alleviates PCIe bottlenecks within each machine, while HaSyn (§5.2.3) reduces network traffic across machines. To better understand the contributions of these two components, we further break down their individual impact. Figure 18(b) shows the size of training data transferred between CPU and GPU, before and after applying FaBC, denoted as Original Data Size and w-FaBC, respectively. On average, FaBC reduces intra-machine data transfer volume by 53.1%, demonstrating its effectiveness in relieving PCIe overhead. Finally, Figure 18(c) compares HaSyn against the conventional full synchronization method AllSync adopted by systems like DistDGL and PBG. The results show that HaSyn reduces communication cost by over 90% during each synchronization period. We compare HaSyn with AllSync and RandomSync on *LJ*.



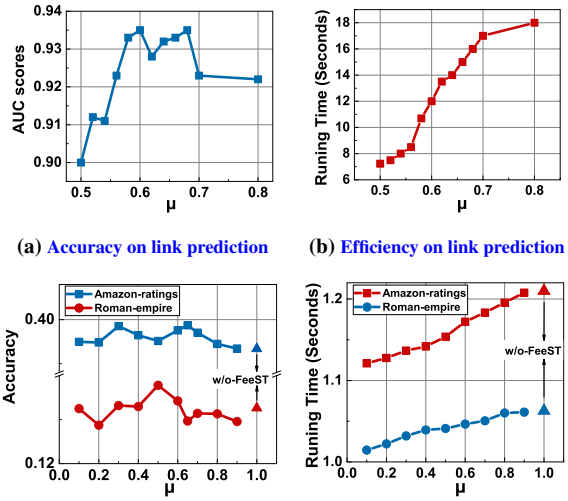
(a) Decouple (b) Pipeline (c) Breakdown Analysis

Figure 19: Resource utilization and runtime breakdown analysis.



(a) Sampling (b) Training

Figure 20: Breakdown analysis of FeLoG for sampling and training, and impact of FeeST on efficiency.



(a) Accuracy on link prediction (b) Efficiency on link prediction

(c) Accuracy on node classification (d) Efficiency on node classification

Figure 21: Sensitivity of FeeST w.r.t. parameter μ.

Compared with AllSync, HaSyn reduces runtime from 509.39s to 111.26s, achieving a 4.58× speedup with only a slight AUC drop. It also achieves higher AUC than RandomSync under the same synchronization size, indicating that update hotness is effective for selecting embeddings to synchronize. In addition, we assess how compression frequency range affects the compression ratio of FaBC in § 6.7. R2.E1.3

Computing Resources Utilization. To improve computational efficiency, FeLoG introduces a round-interleaved pipeline mechanism (RiPP, §5.3) by decoupling the sampling and training operators. To evaluate its effectiveness, we measure CPU and GPU utilization over time on the *LJ* dataset under both decoupled and pipelined execution. As shown in Figure 19(a), under decoupled execution, CPU utilization drops significantly after the initial sampling phase, leaving resources idle. Similarly, GPU utilization remains low, as each training round must wait for sampling to complete. In contrast, RiPP enables concurrent execution between rounds; the sampling stage overlaps with the training stage through different threads and devices. This design leads to much higher resource utilization: Figures 19(b) show average CPU and GPU utilizations of 82.2% and 79.2%, respectively. Overall, RiPP effectively boosts system throughput and scalability by maximizing hardware utilization.

Breakdown Analysis We further provide a fine-grained runtime breakdown of FeLoG. As shown in Figure 19(c), we separate the execution into sampling (FeeST-S), training (FeeST-T), active-set computation (FeeST-A), compression (FaBC-C), decompression (FaBC-D), embedding synchronization (HaSyn-S), and synchronization-set selection (HaSyn-C) across different graphs. The normalized breakdown shows that sampling and training remain the dominant components. In contrast, FeeST-A, FaBC, and HaSyn introduce only limited runtime overhead, accounting for 4.0%, 4.7%, and 2.4% on average, respectively. In addition, the dominant extra memory of HaSyn comes from the synchronization-set buffer, whose size is $|\mathcal{F}| \times d \times 4$ bytes for d-dimensional float embeddings. For example, on *LJ*, $|\mathcal{F}| = 1316$, so it requires only 0.64 MB memory footprint. R2.E2.1 R1.W1

6.7 Experiments on Parameter Sensitivity Analysis

To further assess the sensitivity of FeeST to the similarity convergence parameter μ, we report the *AUC* scores and execution times of FeLoG on the *Flickr* graph with varying μ values. While a smaller μ often requires more sampling to reach convergence, increasing μ leads to longer training time without consistently improving accuracy. As shown in Figure 21(a) and (b), the *AUC* score peaks around μ = 0.6-0.65, beyond which accuracy slightly drops, while execution time continues to grow. This observation suggests that μ values in this range provide a favorable trade-off between efficiency and accuracy for this dataset. Similar trends are observed on other datasets used in our experiments, so we use $\mu = 0.65$ as the default configuration for our datasets in the experiments. For heterophilic settings, we further evaluate FeeST on *Amazon-ratings* (AR) and *Roman-empire* (RE). As shown in Figure 21 (c) and (d), FeeST provides a meaningful quality-efficiency trade-off under varying μ: *AR* is broadly stable, while *RE* favors a lower threshold. Runtime increases with μ, and $\mu = 1$ corresponds to the w/o-FeeST setting. These results support validation-calibrated μ in heterophilic settings, with only 1.85s and 1.68s per candidate on *AR* and *RE*, respectively. R1.W3 R2.E1.1 R3.D2

To assess how compression frequency range affects the compression ratio, Table 6 shows the results when compressing the top-1, top-2, and top-3 most frequent nodes in the training data. The results indicate that expanding the compression range does not significantly improve the compression ratio. Thus, a balance must be struck between compression efficiency and computation overhead. FeLoG addresses this by analyzing the first-round sampling to dynamically configure compression parameters, effectively balancing compression gain and processing cost.

7 CONCLUSIONS

We presented FeLoG, an scalable and efficient distributed system for graph embedding that addresses inefficiency, high communication overhead, and low resource utilization. FeLoG introduces three innovations: (1) a feedback-coupled sampling-training model

Table 6: Impact of different compression frequency ranges on compression ratio.

Dataset	Top-1	Top-2	Top-3
Flickr	0.67	0.48	0.46
YouTube	0.59	0.42	0.40
LiveJournal	0.59	0.48	0.44
ComOrkut	0.60	0.49	0.43
Twitter	0.59	0.47	0.41

that adapts sampling based on real-time embedding quality, (2) an activity-aware communication mechanism that reduces overhead, and (3) a round-interleaved pipeline for improved CPU-GPU utilization. Our experiments on billion-edge graphs demonstrate that FeLoG significantly improves system efficiency, scalability, and generalizability over state-of-the-art systems, highlighting the potential of feedback loop design for large-scale machine learning workloads.

R3.D3 Future work includes incorporating topology-aware optimization for intra-node GPU interconnects and exploring GPU-directed remote data access to further reduce CPU-mediated transfers.

REFERENCES

[1] 2026. Our code and datasets. https://github.com/RocmFang/FeLoG.

[2] Xin Ai, Qiange Wang, Chunyu Cao, Yanfeng Zhang, Chaoyi Chen, Hao Yuan, Yu Gu, and Ge Yu. 2024. NeutronOrch: Rethinking Sample-Based GNN Training under CPU-GPU Heterogeneous Environments. *Proceedings of the VLDB Endowment* 17, 8 (2024), 1995–2008.

[3] Xin Ai, Hao Yuan, Zeyu Ling, Qiange Wang, Yanfeng Zhang, Zhenbo Fu, Chaoyi Chen, Yu Gu, and Ge Yu. 2025. NeutronTP: Load-Balanced Distributed Full-Graph GNN Training with Tensor Parallelism. In *51st International Conference on Very Large Data Bases*. 173–186.

[4] Guangji Bai, Ziyang Yu, Zheng Chai, Yue Cheng, and Liang Zhao. 2025. Staleness-alleviated distributed gnn training via online dynamic-embedding prediction. In *Proceedings of the 2025 SIAM International Conference on Data Mining (SDM)*. SIAM, 578–587.

[5] Albert-László Barabási and Réka Albert. 1999. Emergence of scaling in random networks. *Science* 286, 5439 (1999), 509–512.

[6] Paolo Boldi, Massimo Santini, and Sebastiano Vigna. 2008. A large time-aware web graph. In *ACM SIGIR Forum*, Vol. 42. ACM New York, NY, USA, 33–38.

[7] Hongyun Cai, Vincent W Zheng, and Kevin Chen-Chuan Chang. 2018. A comprehensive survey of graph embedding: Problems, techniques, and applications. *IEEE Transactions on Knowledge and Data Engineering* 30, 9 (2018), 1616–1637.

[8] Chunyu Cao, Xin Ai, Qiange Wang, Yanfeng Zhang, Zhenbo Fu, Hao Yuan, Mingyi Cao, Chaoyi Chen, Yingyou Wen, Yu Gu, et al. 2025. NeutronHeter: Optimizing Distributed Graph Neural Network Training for Heterogeneous Clusters. *Proceedings of the ACM on Management of Data* 3, 4 (2025), 1–29.

[9] Jiahang Cao, Jinyuan Fang, Zaiqiao Meng, and Shangsong Liang. 2024. Knowledge graph embedding: A survey from the perspective of representation spaces. *Comput. Surveys* 56, 6 (2024), 1–42.

[10] Deepayan Chakrabarti, Yiping Zhan, and Christos Faloutsos. 2004. R-MAT: A Recursive Model for Graph Mining. In *Proceedings of the SIAM International Conference on Data Mining*. 442–446.

[11] Chee-Yong Chan and Yannis E Ioannidis. 1998. Bitmap index design and evaluation. In *Proceedings of the 1998 ACM SIGMOD International Conference on Management of Data*. 355–366.

[12] Jie Chen, Tengfei Ma, and Cao Xiao. 2018. FastGCN: Fast Learning with Graph Convolutional Networks via Importance Sampling. In *International Conference on Learning Representations*.

[13] Weijian Chen, Shuibing He, Haoyang Qu, and Xuechen Zhang. 2025. LeapGNN: Accelerating Distributed GNN Training Leveraging Feature-Centric Model Migration. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*. 255–270.

[14] Xu Cheng, Liang Yao, Feng He, Yukuo Cen, Yufei He, Chenhui Zhang, Wenzheng Feng, Hongyun Cai, and Jie Tang. 2025. LPS-GNN: Deploying Graph Neural Networks on Graphs with 100-Billion Edges. *arXiv preprint arXiv:2507.14570* (2025).

[15] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitansky, Robert Osazuwa Ness, and Jonathan Larson. 2024. From local to global: A graph rag approach to query-focused summarization. *arXiv preprint arXiv:2404.16130* (2024).

[16] R. Fan, K. Chang, C. Hsieh, X. Wang, and C. Lin. 2008. LIBLINEAR: A Library for Large Linear Classification. *Journal of Machine Learning Research* (2008), 1871–1874.

[17] Peng Fang, Arijit Khan, Siqiang Luo, Fang Wang, Dan Feng, Zhenli Li, Wei Yin, and Yuchao Cao. 2023. Distributed Graph Embedding with Information-Oriented Random Walks. *Proceedings of the VLDB Endowment* 16, 7 (2023), 1643–1656.

[18] Peng Fang, Zhenli Li, Arijit Khan, Siqiang Luo, Fang Wang, Zhan Shi, and Dan Feng. 2025. Information-Oriented Random Walks and Pipeline Optimization for Distributed Graph Embedding. *IEEE Transactions on Knowledge and Data Engineering* 37, 1 (2025), 408–422.

[19] Peng Fang, Siqiang Luo, Fang Wang, Bolong Zheng, Hong Jiang, Dan Feng, Hechang Pan, and Xingyu Wan. 2025. OMeGa: Boosting Large-Scale Graph Embeddings with Heterogeneous Memory Processing. In *2025 IEEE 41st International Conference on Data Engineering*. 3369–3383.

[20] Peng Fang, Fang Wang, Zhan Shi, Hong Jiang, Dan Feng, Xianghao Xu, and Wei Yin. 2022. How to Realize Efficient and Scalable Graph Embeddings via an Entropy-driven Mechanism. *IEEE Transactions on Big Data* (2022).

[21] Peng Fang, Fang Wang, Zhan Shi, Hong Jiang, Dan Feng, and Lei Yang. 2021. HuGE: An entropy-driven approach to efficient and scalable graph embeddings. In *IEEE 37th International Conference on Data Engineering*. IEEE, 2045–2050.

[22] Tong Geng, Ang Li, Runbin Shi, Chunshu Wu, Tianqi Wang, Yanfei Li, Pouya Haghi, Antonino Tumeo, Shuai Che, Steve Reinhardt, et al. 2020. AWB-GCN: A graph convolutional network accelerator with runtime workload rebalancing. In *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 922–936.

[23] Chenghua Gong, Yao Cheng, Jianxiang Yu, Can Xu, Caihua Shan, Siqiang Luo, and Xiang Li. 2024. A survey on learning from graphs with heterophily: Recent advances and future directions. *arXiv preprint arXiv:2401.09769* (2024).

[24] Aditya Grover and Jure Leskovec. 2016. node2vec: Scalable feature learning for networks. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 855–864.

[25] Will Hamilton, Zhitao Ying, and Jure Leskovec. 2017. Inductive Representation Learning on Large Graphs. In *Advances in Neural Information Processing Systems*, Vol. 30.

[26] Jiafeng Hu, Reynold Cheng, Zhipeng Huang, Yixang Fang, and Siqiang Luo. 2017. On embedding uncertain graphs. In *Proceedings of the 2017 ACM on Conference on Information and Knowledge Management*. 157–166.

[27] Weihua Hu, Matthias Fey, Marinka Zitnik, Yuxiao Dong, Hongyu Ren, Bowen Liu, Michele Catasta, and Jure Leskovec. 2020. Open graph benchmark: Datasets for machine learning on graphs. *Advances in neural information processing systems* 33 (2020), 22118–22133.

[28] Shihao Ji, Nadathur Satish, Sheng Li, and Pradeep K Dubey. 2019. Parallelizing word2vec in shared and distributed memory. *IEEE Transactions on Parallel and Distributed Systems* 30, 9 (2019), 2090–2100.

[29] M. M. Keikha, M. Rahgozar, and M. Asadpour. 2018. Community Aware Random Walk for Network Embedding. *Knowledge-Based Systems* 148 (2018), 47–54.

[30] Thomas N Kipf and Max Welling. 2017. Semi-Supervised Classification with Graph Convolutional Networks. In *International Conference on Learning Representations*.

[31] Haewoon Kwak, Changhyun Lee, Hosung Park, and Sue Moon. 2010. What is Twitter, a social network or a news media?. In *Proceedings of the 19th International Conference on World Wide Web*. 591–600.

[32] Yunjae Lee, Jinha Chung, and Minsoo Rhu. 2022. Smartsage: training large-scale graph neural networks using in-storage processing architectures. In *Proceedings of the 49th Annual International Symposium on Computer Architecture*. 932–945.

[33] Adam Lerer, Ledell Wu, Jiajun Shen, Timothee Lacroix, Luca Wehrstedt, Abhijit Bose, and Alex Peysakhovich. 2019. Pytorch-biggraph: A large scale graph embedding system. In *Proceedings of Machine Learning and Systems*, Vol. 1. 120–131.

[34] Jure Leskovec, Daniel Huttenlocher, and Jon Kleinberg. 2010. Predicting positive and negative links in online social networks. In *Proceedings of the 19th International Conference on World Wide Web*. 641–650.

[35] Yongkun Li, Zhiyong Wu, Shuai Lin, Hong Xie, Min Lv, Yinlong Xu, and John CS Lui. 2019. Walking with perception: Efficient random walk sampling via common neighbor awareness. In *2019 IEEE 35th International Conference on Data Engineering*. IEEE, 962–973.

[36] Zhiyuan Li, Xun Jian, Yue Wang, Yingxia Shao, and Lei Chen. 2024. Daha: Accelerating gnn training with data and hardware aware execution planning. *Proceedings of the VLDB Endowment* 17, 6 (2024), 1364–1376.

[37] Ningyi Liao, Dingheng Mo, Siqiang Luo, Xiang Li, and Pengcheng Yin. 2022. SCARA: scalable graph neural networks with feature-oriented optimization. *arXiv preprint arXiv:2207.09179* (2022).

[38] Dongkyun Lim and John Kim. 2025. TidalMesh: Topology-Driven AllReduce Collective Communication for Mesh Topology. In *2025 IEEE International Symposium on High Performance Computer Architecture*. IEEE, 1526–1540.

[39] Tianfeng Liu, Yangrui Chen, Dan Li, Chuan Wu, Yibo Zhu, Jun He, Yanghua Peng, Hongzheng Chen, Hongzhi Chen, and Chuanxiong Guo. 2023. BGL:

GPU-Efficient GNN training by optimizing graph data I/O and preprocessing. In *20th USENIX Symposium on Networked Systems Design and Implementation*. 103–118.

[40] Linhao Luo, Zicheng Zhao, Gholamreza Haffari, Dinh Phung, Chen Gong, and Shirui Pan. 2025. GFM-RAG: graph foundation model for retrieval augmented generation. *arXiv preprint arXiv:2502.01113* (2025).

[41] Siqiang Luo, Zichen Zhu, Xiaokui Xiao, Yin Yang, Chunbo Li, and Ben Kao. 2023. Multi-Task Processing in Vertex-Centric Graph Systems: Evaluations and Insights. In *International Conference on Extending Database Technology*. 247–259.

[42] Miller McPherson, Lynn Smith-Lovin, and James M Cook. 2001. Birds of a feather: Homophily in social networks. *Annual review of sociology* 27, 1 (2001), 415–444.

[43] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. 2013. Efficient estimation of word representations in vector space. In *International Conference on Learning Representations*.

[44] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S Corrado, and Jeff Dean. 2013. Distributed Representations of Words and Phrases and their Compositionality. *Advances in Neural Information Processing Systems* 26 (2013).

[45] Jeongmin Brian Park, Vikram Sharma Mailthody, Zaid Qureshi, and Wen-mei Hwu. 2024. Accelerating Sampling and Aggregation Operations in GNN Frameworks with GPU Initiated Direct Storage Accesses. *Proceedings of the VLDB Endowment* 17, 6 (2024), 1227–1240.

[46] Yeonhong Park, Sunhong Min, and Jae W Lee. 2022. Ginex: SSD-Enabled Billion-Scale Graph Neural Network Training on a Single Machine via Provably Optimal in-Memory Caching. *Proceedings of the VLDB Endowment* 15, 11 (2022), 2626–2639.

[47] Bryan Perozzi, Rami Al-Rfou, and Steven Skiena. 2014. DeepWalk: Online Learning of Social Representations. In *Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 701–710.

[48] Oleg Platonov, Denis Kuznedelev, Artem Babenko, and Liudmila Prokhorenkova. 2023. Characterizing graph datasets for node classification: Homophily-heterophily dichotomy and beyond. *Advances in Neural Information Processing Systems* 36 (2023), 523–548.

[49] Oleg Platonov, Denis Kuznedelev, Michael Diskin, Artem Babenko, and Liudmila Prokhorenkova. 2023. A Critical Look at the Evaluation of GNNs under Heterophily: Are We Really Making Progress?. In *International Conference on Learning Representations*.

[50] Linshan Qiu, Lu Chen, Hailiang Jie, Xiangyu Ke, Yunjun Gao, Yang Liu, and Zetao Zhang. 2024. GPU-Accelerated Batch-Dynamic Subgraph Matching. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, 3204–3216.

[51] Alexander Renz-Wieland, Rainer Gemulla, Zoi Kaoudi, and Volker Markl. 2022. NuPS: A Parameter Server for Machine Learning with Non-Uniform Parameter Access. In *Proceedings of the 2022 International Conference on Management of Data*. 481–495.

[52] Andrea Rossi, Denilson Barbosa, Donatella Firmani, Antonio Matinata, and Paolo Merialdo. 2021. Knowledge graph embedding for link prediction: A comparative analysis. *ACM Transactions on Knowledge Discovery from Data* 15, 2 (2021), 1–49.

[53] Guangming Sheng, Junwei Su, Chao Huang, and Chuan Wu. 2024. Mspipe: Efficient temporal gnn training via staleness-aware pipeline. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 2651–2662.

[54] Jaeyong Song, Hongsun Jang, Hunseong Lim, Jaewon Jung, Youngsok Kim, and Jinho Lee. 2024. GraNNDis: Fast distributed graph neural network training framework for multi-server clusters. In *Proceedings of the 2024 International Conference on Parallel Architectures and Compilation Techniques*. 91–107.

[55] Dahai Tang, Jiali Wang, Rong Chen, Lei Wang, Wenyuan Yu, Jingren Zhou, and Kenli Li. 2024. Xgnn: Boosting multi-gpu gnn training via global gnn memory store. *Proceedings of the VLDB Endowment* 17, 5 (2024), 1105–1118.

[56] Lei Tang and Huan Liu. 2009. Scalable learning of collective behavior based on sparse social dimensions. In *Proceedings of the 18th ACM Conference on Information and Knowledge Management*. 1107–1116.

[57] Anton Tsitsulin, Davide Mottin, Panagiotis Karras, and Emmanuel Müller. 2018. Verse: Versatile graph embeddings from similarity measures. In *Proceedings of the 2018 World Wide Web conference*. 539–548.

[58] Ke Tu, Peng Cui, Xiao Wang, Philip S Yu, and Wenwu Zhu. 2018. Deep recursive network embedding with regular equivalence. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 2357–2366.

[59] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. 2018. Graph Attention Networks. In *International Conference on Learning Representations*.

[60] Roger Waleffe, Jason Mohoney, Theodoros Rekatsinas, and Shivaram Venkataraman. 2023. MariusGNN: Resource-Efficient Out-of-Core Training of Graph Neural Networks. In *Proceedings of the 18th European Conference on Computer Systems* (Rome, Italy). 144–161.

[61] Cheng Wan, Youjie Li, Cameron R. Wolfe, Anastasios Kyrillidis, Nam Sung Kim, and Yingyan Lin. 2022. PipeGCN: Efficient Full-Graph Training of Graph Convolutional Networks with Pipelined Feature Communication. In *International Conference on Learning Representations*.

[62] Daixin Wang, Peng Cui, and Wenwu Zhu. 2016. Structural deep network embedding. In *Proceedings of the 22nd ACM SIGKDD international conference on Knowledge discovery and data mining*. 1225–1234.

[63] Hongwei Wang, Jia Wang, Jialin Wang, Miao Zhao, Weinan Zhang, Fuzheng Zhang, Xing Xie, and Minyi Guo. 2018. GraphGAN: Graph Representation Learning With Generative Adversarial Nets. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 32.

[64] Minjie Wang, Da Zheng, Zihao Ye, Quan Gan, Mufei Li, Xiang Song, Jinjing Zhou, Chao Ma, Lingfan Yu, Yu Gai, et al. 2019. Deep graph library: A graph-centric, highly-performant package for graph neural networks. *arXiv preprint arXiv:1909.01315* (2019).

[65] Qiange Wang, Yanfeng Zhang, Hao Wang, Chaoyi Chen, Xiaodong Zhang, and Ge Yu. 2022. Neutronstar: Distributed GNN Training with Hybrid Dependency Management. In *Proceedings of the 2022 International Conference on Management of Data*. 1301–1315.

[66] Xiao Wang, Deyu Bo, Chuan Shi, Shaohua Fan, Yanfang Ye, and Philip S Yu. 2022. A survey on heterogeneous graph embedding: methods, techniques, applications and sources. *IEEE Transactions on Big Data* 9, 2 (2022), 415–436.

[67] Xuhong Wang, Ding Lyu, Mengjian Li, Yang Xia, Qi Yang, Xinwen Wang, Xinguang Wang, Ping Cui, Yupu Yang, Bowen Sun, et al. 2021. Apan: Asynchronous propagation attention network for real-time temporal graph embedding. In *Proceedings of the 2021 International Conference on Management of Data*. 2628–2638.

[68] Yuke Wang, Boyuan Feng, Zheng Wang, Tong Geng, Kevin Barker, Ang Li, and Yufei Ding. 2023. MGG: Accelerating graph neural networks with Fine-Grained Intra-Kernel Communication-Computation pipelining on Multi-GPU platforms. In *17th USENIX Symposium on Operating Systems Design and Implementation*. 779–795.

[69] Yuyue Wang, Xiurui Pan, Yuda An, Jie Zhang, and Glenn Reinman. 2024. BeaconGNN: Large-Scale GNN Acceleration with Out-of-Order Streaming In-Storage Computing. In *IEEE International Symposium on High-Performance Computer Architecture*. IEEE, 330–344.

[70] Zesong Wang, Peng Fang, Fang Wang, Hong Jiang, Yimin Lu, Zhan Shi, and Dan Feng. 2025. SpiderCache: Semantic-Aware Caching Strategy for DNN Training. In *Proceedings of the 54th International Conference on Parallel Processing (ICPP'25)*. 320–330.

[71] Brian J Worton. 1989. Kernel methods for estimating the utilization distribution in home-range studies. *Ecology* 70, 1 (1989), 164–168.

[72] Qitian Wu, Wentao Zhao, Zenan Li, David P Wipf, and Junchi Yan. 2022. Nodeformer: A scalable graph structure learning transformer for node classification. *Advances in Neural Information Processing Systems* 35 (2022), 27387–27401.

[73] Yidi Wu, Kaihao Ma, Zhenkun Cai, Tatiana Jin, Boyang Li, Chenguang Zheng, James Cheng, and Fan Yu. 2021. Seastar: Vertex-centric programming for graph neural networks. In *Proceedings of the 16th European Conference on Computer Systems*. 359–375.

[74] Jaewon Yang and Jure Leskovec. 2012. Defining and evaluating network communities based on ground-truth. In *Proceedings of the ACM SIGKDD Workshop on Mining Data Semantics*. 1–8.

[75] Renchi Yang, Jieming Shi, Xiaokui Xiao, Yin Yang, and Sourav S Bhowmick. 2020. Homogeneous Network Embedding for Massive Graphs via Reweighted Personalized PageRank. *Prococeding of VLDB Endowment* 13, 5 (2020), 670–683.

[76] Tianbao Yang and Yiming Ying. 2022. AUC maximization in the era of big data and AI: A survey. *ACM computing surveys* 55, 8 (2022), 1–37.

[77] Zihao Yu, Ningyi Liao, and Siqiang Luo. 2024. GENTI: GPU-powered Walk-based Subgraph Extraction for Scalable Representation Learning on Dynamic Graphs. *Proceedings of the VLDB Endowment* 17, 9 (2024), 2269–2278.

[78] Reza Zafarani and Huan Liu. 2009. Social Computing Data Repository at ASU. In *http://socialcomputing.asu.edu*.

[79] Jie Zhang, Yuxiao Dong, Yan Wang, Jie Tang, and Ming Ding. 2019. ProNE: Fast and Scalable Network Representation Learning. In *International Joint Conference on Artificial Intelligence*, Vol. 19. 4278–4284.

[80] J. Zhang, Z. Zhang, Z. Wang, Y. Liu, and L. Deng. 2018. Ontological function annotation of long non-coding RNAs through hierarchical multi-label classification. *Bioinformatics* 34, 10 (2018), 1750–1757.

[81] M. Zhang and Z. Zhou. 2006. Multilabel neural networks with applications to functional genomics and text categorization. *IEEE Transactions on Knowledge and Data Engineering* 18, 10 (2006), 1338–1351.

[82] Qinggang Zhang, Shengyuan Chen, Yuanchen Bei, Zheng Yuan, Huachi Zhou, Zijin Hong, Junnan Dong, Hao Chen, Yi Chang, and Xiao Huang. 2025. A survey of graph retrieval-augmented generation for customized large language models. *arXiv preprint arXiv:2501.13958* (2025).

[83] Shijie Zhang, Hongzhi Yin, Tong Chen, Zi Huang, Lizhen Cui, and Xiangliang Zhang. 2021. Graph embedding for recommendation against attribute inference

attacks. In *Proceedings of the Web Conference 2021*. 3002–3014.

[84] Chenguang Zheng, Hongzhi Chen, Yuxuan Cheng, Zhezheng Song, Yifan Wu, Changji Li, James Cheng, Hao Yang, and Shuai Zhang. 2022. ByteGNN: Efficient Graph Neural Network Training at Large Scale. *Proceedings of VLDB Endowment* 15, 6 (2022), 1228–1242.

[85] Da Zheng, Chao Ma, Minjie Wang, Jinjing Zhou, Qidong Su, Xiang Song, Quan Gan, Zheng Zhang, and George Karypis. 2020. DistDGL: Distributed Graph Neural Network Training for Billion-Scale Graphs. In *2020 IEEE/ACM 10th Workshop on Irregular Applications: Architectures and Algorithms (IA3)*. IEEE, 36–44.

[86] Liu Ziyin, Kangqiao Liu, Takashi Mori, and Masahito Ueda. 2022. STRENGTH OF MINIBATCH NOISE IN SGD. In *10th International Conference on Learning Representations, ICLR 2022*.